Parte II Il ciclo di vita del software

CONTENUTI

Capitolo 9	Ingegneria dei requisiti	**213**
Capitolo 10	Modellazione	**261**
Capitolo 11	Architettura del software	**289**
Capitolo 12	Progettazione del software	**325**
Capitolo 13	Test del software	**405**
Capitolo 14	Manutenzione del software	**465**
Capitolo 15	Strumenti software	**507**

Quando progetti un giardino, inizi formulando le tue esigenze: quanto dovrebbe essere grande l'area erbosa? dovresti lasciare un angolo per allevare patate? dove dovrebbe essere messo il cestino della sabbia? i requisiti interferiscono con futuri lavori di manutenzione della casa (!)? ecc. Successivamente, viene redatto un progetto, che viene accuratamente documentato in un progetto. Solo allora il giardiniere taglierà la prima zolla.

Un approccio simile viene seguito durante lo sviluppo del software. In una serie di fasi – ingegneria dei requisiti, progettazione, implementazione e test – il sistema software prenderà forma. Dopo che il software è stato consegnato al cliente, deve essere mantenuto. La reiterazione delle fasi si verifica perché le modifiche devono essere incorporate e gli errori devono essere corretti. Il risultato è un processo altamente ciclico, il "ciclo di vita del software".

Le varie fasi del ciclo di sviluppo iniziale sono gli argomenti dei capitoli 9 – 13 e il capitolo 14 è dedicato alla manutenzione del software. In ogni fase vengono utilizzate tecniche di modellazione per rappresentare i risultati di quella fase. Un esempio di tecniche di modellazione note è discusso nel Capitolo 10 . In ogni fase, inoltre, vengono impiegati strumenti. Le principali classi di strumenti e il loro ruolo nel processo di sviluppo del software sono discusse nel Capitolo 15 .

9 Ingegneria dei requisiti

OBIETTIVI FORMATIVI

- Comprendere che l'ingegneria dei requisiti è un processo ciclico che coinvolge quattro tipi di attività: elicitazione, specificazione, convalida e negoziazione
- Apprezzare il ruolo delle problematiche sociali e cognitive nell'ingegneria dei requisiti
- Essere in grado di distinguere un certo numero di tecniche di elicitazione dei requisiti
- Essere a conoscenza del contenuto di un documento di specifica dei requisiti
- Conoscere varie tecniche e notazioni per specificare i requisiti
- Conoscere diversi modi per strutturare un insieme di requisiti

Questo capitolo tratta l'ingegneria dei requisiti, la prima fase importante di un progetto di sviluppo software. L'aspetto più impegnativo e difficile dell'ingegneria dei requisiti è ottenere una descrizione completa del problema da risolvere. Discutiamo una serie di tecniche per suscitare i requisiti dall'utente. Dopo l'elicitazione, questi requisiti devono essere negoziati, convalidati e documentati.

- La parte più difficile della costruzione di un sistema è decidere cosa costruire.
  Brooks (**1987**)

La fase di ingegneria dei requisiti è il primo passo importante verso la soluzione di un problema di elaborazione dei dati. In questa fase vengono accuratamente individuate e documentate le esigenze dell'utente rispetto al futuro sistema. Questi requisiti riguardano sia le funzioni da fornire che una serie di requisiti aggiuntivi, come prestazioni, affidabilità, documentazione per l'utente, formazione dell'utente, costi e così via. Durante la fase di ingegneria dei requisiti, non affrontiamo ancora la questione di *come* soddisfare questi requisiti dell'utente in termini di componenti di sistema e loro interazione. Questo è rinviato alla fase di progettazione.

Un requisito è "una condizione o capacità necessaria a un utente per risolvere un problema o raggiungere un obiettivo" (IEEE610, 1990). L'"utente" a cui si fa riferimento in questa definizione può essere un utente finale del sistema, una persona dietro lo schermo. Tuttavia, può anche denotare diverse classi di utenti indiretti, come le persone che non ruotano le manopole ma utilizzano piuttosto le informazioni fornite dal sistema. Può anche indicare il cliente (cliente) che paga la fattura. Durante l'ingegneria dei requisiti, diversi tipi di utenti possono essere fonte di diversi tipi di requisiti. Si spera che gli utenti finali siano la principale fonte di informazioni sui requisiti funzionali e relativi alle attività. Altri requisiti, ad esempio quelli relativi a problemi di sicurezza, potrebbero essere formulati da altre parti interessate.

La parola "requisito" suggerisce che, una volta affermato, **deve** essere soddisfatto. In realtà, non è quasi mai così. La maggior parte dei requisiti è negoziabile. Il time to market, i costi, i requisiti di qualità contrastanti e le esigenze contrastanti delle parti interessate portano tutti a una situazione in cui potrebbe essere necessario fare dei compromessi.

Il risultato della fase di ingegneria dei requisiti è documentato nella **specifica dei requisiti.** La specifica dei requisiti riflette la comprensione reciproca del problema da risolvere tra l'analista e il cliente. È la base per un contratto, formale o informale, tra il cliente del sistema e l'organizzazione di sviluppo. Infine, il sistema consegnato sarà valutato verificando la sua conformità alla specifica dei requisiti.

La specifica dei requisiti funge da punto di partenza per la fase di progettazione. In fase di progettazione, l'architettura del sistema è concepita in termini di componenti di sistema e interfacce tra tali componenti. La fase di progettazione si traduce anche in una specifica: una descrizione precisa – preferibilmente in un linguaggio formale – dell'architettura, dei suoi componenti e delle sue interfacce.

La nozione di "specificazione" ha quindi diversi significati. Per evitare confusione, utilizzeremo sempre il prefisso 'requisiti' se denota il risultato della fase di ingegneria dei requisiti. A peggiorare le cose, la fase in cui i requisiti dell'utente vengono analizzati e documentati è talvolta chiamata anche fase di "specificazione". Riteniamo che questo sia in qualche modo un termine improprio e non useremo il termine in questo modo.

Usiamo il termine **ingegneria dei requisiti** piuttosto che la nozione più ristretta di **analisi dei requisiti** per sottolineare che si tratta di un processo iterativo e cooperativo di analisi di un problema, documentazione delle osservazioni risultanti e verifica dell'accuratezza della comprensione acquisita. L'ingegneria dei requisiti non riguarda solo questioni tecniche su come rappresentare i requisiti. Anche gli aspetti sociali e cognitivi giocano un ruolo dominante.

L'ingegneria dei requisiti e la progettazione generalmente non possono essere rigorosamente separate nel tempo. In alcuni casi, la specifica dei requisiti è molto formale e può essere vista come una specifica progettuale di alto livello del sistema da costruire. Spesso, un progetto preliminare viene eseguito dopo che è stata determinata una serie iniziale di requisiti. Sulla base del risultato di questo sforzo di progettazione, le specifiche dei requisiti possono essere modificate e perfezionate. Questo tipo di iterazione si verifica anche quando vengono utilizzate tecniche di prototipazione. Nei progetti di sviluppo puramente agile, i requisiti emergono in concomitanza con un sistema attivo e funzionante. Tecniche ben note come i diagrammi di flusso di dati e i diagrammi di classe UML vengono utilizzate per strutturare e documentare sia le specifiche dei requisiti che i progetti.

È solo per facilità di presentazione che le fasi di progettazione e ingegneria dei requisiti sono rigorosamente separate e trattate consecutivamente in questo libro.

Durante l'ingegneria dei requisiti, vengono affrontate una serie di questioni abbastanza diverse. Prendiamo un esempio e consideriamo il caso (ipotetico) di una biblioteca universitaria che automatizza il suo funzionamento. Iniziamo con la libreria che contiene una serie di armadi. Questi armadietti contengono un numero enorme di carte, una per libro. Ogni

scheda contiene i nomi degli autori, il titolo del libro, l'ISBN, l'anno di pubblicazione e altri dati utili. Le carte sono ordinate alfabeticamente in base al nome del primo autore di ogni libro.

Questo sistema di ordinamento presenta infatti grossi problemi in quanto funziona bene solo se conosciamo il nome del primo autore. Se conosciamo solo il titolo, o se siamo interessati a libri su un determinato argomento, il catalogo dell'autore è di poco o nessun aiuto.

Una soluzione software sembra ovvia. Se memorizziamo i dati per ogni libro una volta in un database, potremmo successivamente ordinare le voci in molti modi diversi. Strumenti appropriati possono consentire all'utente di eseguire ricerche nel database in modo interattivo. Fornendo l'accesso a Internet al database, il servizio può essere notevolmente migliorato.

Durante la fase di ingegneria dei requisiti, verranno sollevati numerosi requisiti degli utenti. Alcuni di questi requisiti riguarderanno l'aggiornamento del database: ovvero l'aggiunta, l'eliminazione e la modifica dei record. Altri riguarderanno funzioni da fornire ai membri ordinari della biblioteca, quali:

- dare un elenco di tutti i libri scritti da X;
- fornire un elenco di tutti i libri il cui titolo contiene Y;
- fornire un elenco di tutti i libri sull'argomento Z;
- dando un elenco di tutti i libri arrivati dopo la data D.

È opportuno cercare di raggruppare i requisiti degli utenti in alcune categorie, che vanno dai "requisiti essenziali" alle "caratteristiche interessanti". Come notato nel Capitolo 3 , gli utenti tendono ad avere difficoltà nell'articolare i propri bisogni reali. È probabile, quindi, che molto sforzo venga speso per realizzare caratteristiche che in seguito si riveleranno semplici campane e fischietti. Utilizzando uno schema a strati sia nella formulazione dei requisiti dell'utente che nella loro successiva realizzazione, è possibile aggirare alcuni dei problemi che affliggono i progetti di sviluppo software. Nel nostro esempio di sistema bibliotecario, ad esempio, il requisito "Dai un elenco di tutti i libri che sono arrivati dopo la data D" potrebbe essere classificato come una caratteristica interessante. Il servizio non viene gravemente degradato se questa funzione non viene fornita, poiché potremmo posizionare temporaneamente le acquisizioni su uno scaffale dedicato.

È anche possibile provare a prevedere una serie di requisiti futuri, che non saranno implementati nel presente progetto. È tuttavia opportuno prestare attenzione a queste questioni in una fase iniziale, in modo che possano essere adattate durante la progettazione del sistema. Eventuali futuri requisiti del nostro sistema bibliotecario potrebbero includere cose come:

- memorizzare informazioni sui libri che sono stati ordinati ma non sono stati ricevuti;
- memorizzare informazioni sui membri della biblioteca, come il loro nome e indirizzo, e le date in cui i libri vengono loro prestati, che possono quindi essere utilizzate per generare un avviso di promemoria per i libri non restituiti in tempo.

Le funzioni di cui sopra riguardano l'uso del software da parte dei membri della biblioteca e del personale bibliotecario. Ci sono anche altre parti interessate, però. Ad esempio, la direzione della biblioteca potrebbe voler utilizzare il sistema per ottenere informazioni sui profili dei membri al fine di migliorare il processo di acquisizione dei titoli.

Oltre a questi requisiti, che riguardano direttamente le funzioni del software da fornire, durante la fase di ingegneria dei requisiti dovrebbero essere affrontate una serie di altre questioni. Per il nostro esempio di libreria, come minimo, devono essere affrontati i seguenti punti:

- Su quale macchina verrà implementato il sistema e quale sistema operativo verrà utilizzato? Se i dati devono essere archiviati in un DBMS, quale (tipo di) DBMS deve essere utilizzato? Che tipo di accesso deve essere utilizzato e quanti punti di accesso saranno supportati?
- Quali classi di utenti si possono distinguere? Nel nostro esempio, dovranno essere serviti sia il personale della biblioteca che i membri della biblioteca. Che tipo di conoscenza hanno questi utenti? Alcune funzioni del sistema saranno limitate a determinate classi di utenti? I normali membri della biblioteca probabilmente non potranno aggiornare il database o stampare il contenuto del database.
- Qual è la dimensione del database e come dovrebbe crescere nel tempo? Questi fattori influenzano sia la capacità di archiviazione necessaria che gli algoritmi da utilizzare. Per un database contenente diverse migliaia di libri, potrebbe essere sufficiente un algoritmo di ricerca non molto efficiente. Per la Library of Congress, invece, la situazione è molto diversa.
- Quale tempo di risposta dovrebbe offrire il sistema? Una richiesta di ricerca per un determinato libro dovrà ricevere una risposta abbastanza rapidamente. Se l'utente deve aspettare troppo a lungo per una risposta, diventerà insoddisfatto e cercherà direttamente negli scaffali. Le domande correlate riguardano l'interazione tra il tempo di risposta e il numero previsto di sessioni di domande per unità di tempo.
- Quanto costerà un impianto di questo tipo? Nel nostro esempio di libreria, non dovremmo solo prestare attenzione ai costi diretti sostenuti dallo sforzo di sviluppo del software. Non va trascurato il costo della conversione delle informazioni contenute negli attuali schedari in un formato di database adeguato. Questi costi indiretti, meno visibili, potrebbero superare di gran lunga il costo diretto della progettazione e dell'attuazione del nuovo sistema.

Questo esempio relativamente semplice mostra già che non è sufficiente elencare semplicemente i requisiti funzionali del nuovo sistema.

Anche l'ambiente previsto del sistema e la sua interazione con quell'ambiente dovrebbero essere analizzati. Nel nostro esempio, questo riguarda la libreria stessa, per cominciare. Le conseguenze dell'introduzione di un sistema come questo possono essere molto maggiori di quanto sembri a prima vista. Le procedure di lavoro possono cambiare, rendendo necessaria la riqualificazione del personale, cambiamenti nelle funzioni del personale e cambiamenti

nella struttura organizzativa complessiva. Alcuni membri del personale potrebbero anche diventare superflui. La verifica del pagamento delle quote associative potrebbe comportare l'interfacciamento con il sistema finanziario, di proprietà di un altro dipartimento.

In generale, la realizzazione di un sistema automatizzato può avere più che semplici ripercussioni tecniche. Spesso non si presta sufficiente attenzione a queste altre ripercussioni. Il mancato successo di molti progetti di sviluppo software può essere ricondotto a una negligenza di aspetti non tecnici.

In pratica, il processo di ingegneria dei requisiti è spesso più complesso di quanto abbozzato sopra:

- L'automazione ordinaria delle librerie è un dominio relativamente noto, in cui ci si può aspettare che gli utenti siano in grado di articolare le proprie esigenze. Ma supponiamo che il nostro sistema bibliotecario debba anche supportare gli anziani nei loro rapporti con il mondo che li circonda, comprese le notizie quotidiane, le normative governative pertinenti, le informazioni sull'assistenza sanitaria e simili. Per quest'ultimo tipo di supporto, sembra più appropriato un approccio più agile, in cui i requisiti emergono man mano che procediamo, piuttosto che essere sollecitati in anticipo.
- Gran parte del software sviluppato oggi è *guidato dal mercato* piuttosto che guidato dal *cliente.* Ad esempio, invece di sviluppare un sistema per una libreria specifica, potremmo sviluppare un'applicazione di libreria "generica". I requisiti per questa applicazione di libreria generica vengono creati esplorando il dominio della libreria, mentre i compromessi tra i requisiti si basano su considerazioni di mercato, adattamento del prodotto e così via. Potremmo decidere che il nostro sistema non deve affrontare le preoccupazioni della Library of Congress (un mercato troppo piccolo), ma che dovrebbe sicuramente interfacciarsi con il sistema contabile Y, poiché quel sistema è ampiamente utilizzato nei dipartimenti universitari e questo è percepito come un mercato importante per la nostra applicazione bibliotecaria.
- Per vari motivi (costo, time-to-market e qualità), potremmo voler impiegare componenti commerciali pronti all'uso (COTS) nel nostro sistema bibliotecario. Dobbiamo quindi scambiare i nostri requisiti con le possibilità offerte da quei componenti COTS.

Secondo Sommerville (2005), distinguiamo quattro processi nell'ingegneria dei requisiti:

- **Elicitazione dei requisiti** In generale, l'analista dei requisiti non è un esperto del dominio modellato. Attraverso l'interazione con specialisti di dominio, come bibliotecari professionisti, deve costruirsi un modello sufficientemente ricco di quel dominio. Pertanto, l'elicitazione dei requisiti riguarda la *comprensione* del problema. Il fatto che diverse discipline siano coinvolte in questo processo complica le cose. In molti casi, l'analista non è un semplice osservatore esterno del dominio da modellare, ma semplicemente ricavando fatti dagli specialisti del dominio. Potrebbe dover prendere posizione in una lotta di potere o decidere tra esigenze contrastanti, partecipando così

attivamente alla costruzione del dominio di interesse. **La sezione 9.1** discute varie questioni relative e una serie di tecniche utilizzate nell'elicitazione dei requisiti.

- **Specifica dei requisiti** Una volta compreso il problema, è necessario *descriverlo.* Nella **Sezione 9.2 vengono** fornite le linee guida per il contenuto di un documento di specifica dei requisiti. Questo documento descrive il prodotto da consegnare, non il processo di sviluppo. I requisiti del progetto sono descritti nel piano del progetto, discusso nel **Capitolo 2** . La raccolta dei requisiti non solo deve essere documentata, ma anche gestita nel corso di un progetto. Esistono numerose tecniche per specificare i requisiti, che vanno da molto informali (linguaggio naturale) a molto formali (matematici). In questo libro vengono discusse alcune di queste tecniche di modellazione, come quelle di UML nel **Capitolo 10** e le tecniche per specificare i requisiti di qualità nel **Capitolo 6** . Le tecniche di progettazione discusse nel **Capitolo 12** sono spesso utilizzate anche per specificare i requisiti. **La sezione 9.3** si limita alla discussione di alcune tecniche globali per la modellazione dei requisiti.
- **Validazione e verifica dei requisiti** Una volta descritto il problema, le diverse parti coinvolte devono *concordarne* la natura. Dobbiamo accertare che i requisiti corretti siano indicati (convalida) e che questi requisiti siano indicati correttamente (verifica). Alcune tecniche di verifica e validazione che possono essere applicate in questa fase iniziale sono abbozzate nella **Sezione 9.4** .
- **Negoziazione dei requisiti** Di solito, i requisiti devono essere negoziati. A causa di vincoli di tempo o di altri fattori, potrebbe essere necessario effettuare una selezione dall'elenco dei requisiti presentato. Oppure le parti interessate potrebbero avere conflitti che devono essere risolti. Spesso le parti interessate hanno requisiti di qualità contrastanti il cui impatto può essere determinato solo osservando l'architettura del software. Questo è ulteriormente trattato nel **Capitolo 11** .

Ovviamente, questi processi implicano iterazioni e feedback. Negli approcci basati sui documenti, queste iterazioni precedono la progettazione e l'implementazione. Nei processi agili, la progettazione e l'implementazione fanno parte del ciclo di iterazione e feedback. In entrambi i casi, esiste un repository centrale in cui sono documentati i requisiti. Le interazioni principali sono mostrate nella Figura 9.1 .

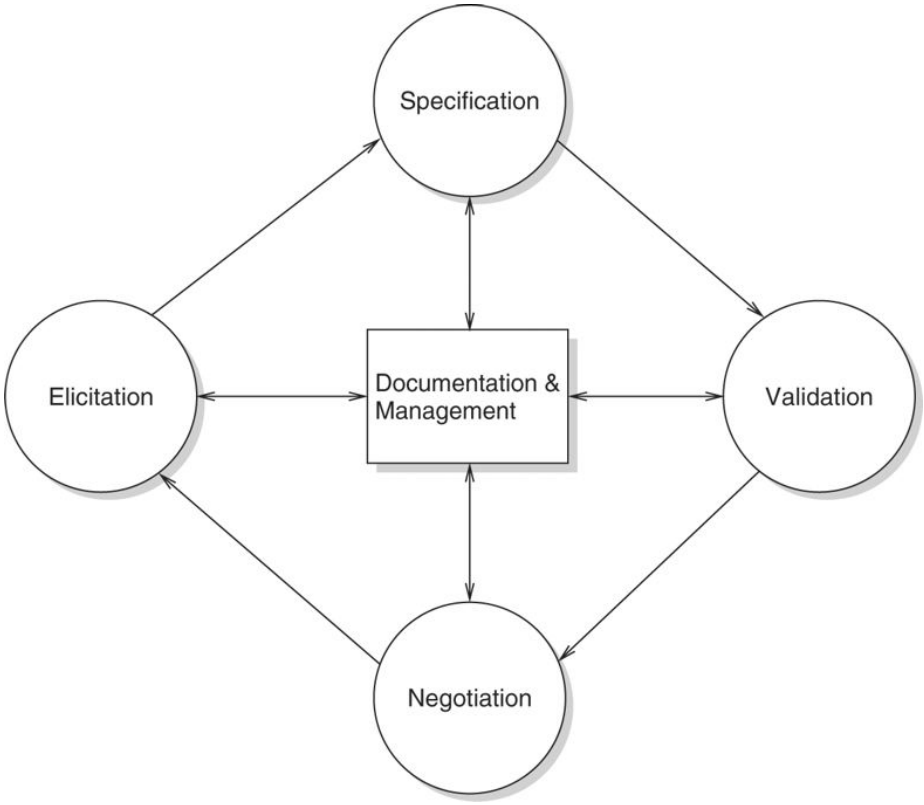


Figura 9.1Un framework per il processo di ingegneria dei requisiti (adattato da (Sommerville, 2005))

L'enfasi nella nostra discussione sull'ingegneria dei requisiti sarà sulla modellazione del comportamento esterno del sistema, cioè tutte quelle parti e aspetti del sistema che *gli utenti finali* considerano importanti. Sono rilevanti anche altri punti di vista, ad esempio un modello che evidenzia il modo in cui il sistema supporta l'azienda o che indica come un sistema viene implementato su una raccolta di dispositivi hardware. Alcuni di questi altri punti di vista sono discussi nel Capitolo 11 .

9.1 ELICITAZIONE DEI REQUISITI

Nel Capitolo 1 , la prima parte del ciclo di vita del software è stata illustrata come mostrato nella Figura 9.2 . Il fatto che il testo 'specificazione dei requisiti' sia inserito in un rettangolo suggerisce, non ingiustamente, che si tratti di qualcosa di molto concreto ed esplicito. Il "problema" è meno ben definito, meno chiaro, persino sfocato in molti casi. L'obiettivo principale della fase di ingegneria dei requisiti è quello di ottenere i contorni ei componenti di questo problema sfocato. Questo processo è anche noto come **modellazione concettuale.**

Durante l'ingegneria dei requisiti stiamo modellando parte della realtà. La parte della realtà a cui siamo interessati è indicata come L'**universo del discorso** (UoD). Esempi di universi di discorso sono un sistema bibliotecario, un sistema di automazione di fabbrica, una catena di montaggio e un sistema di ascensori.

Il modello costruito durante la fase di ingegneria dei requisiti è un **modello concettuale esplicito** dell'UoD. L'aggettivo 'esplicito' indica che il modello deve poter essere comunicato alle persone rilevanti (come analisti e utenti). A tal fine, dovrebbe contenere tutte le informazioni pertinenti dell'UoD. Uno dei problemi persistenti dell'analisi dei requisiti e, del resto, dell'analisi in generale, è quello di tenere conto di tutte le influenze rilevanti e tralasciare i dettagli irrilevanti.

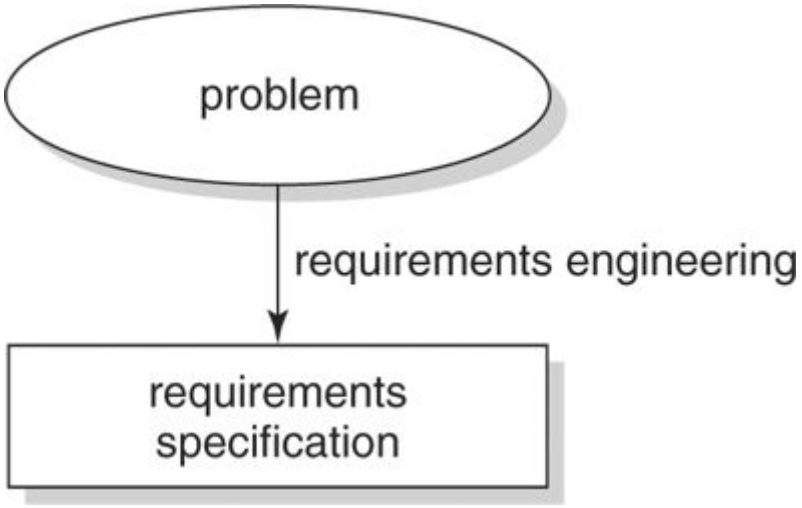


Figura 9.2 La prima parte del ciclo di vita del software

Nel nostro esempio di biblioteca, avremmo potuto facilmente trascurare il fatto che in un certo numero di casi il nome dell'autore come appare sulla copertina di un libro non è il nome dell'autore 'canonico'. Questo fenomeno si verifica in particolare con autori di paesi che utilizzano scritture non latine. La trascrizione del nome russo œexoB recita "Chekhov" in inglese e "Tsjechow" in olandese. In questi casi, i bibliotecari vogliono includere il nome dell'autore due volte: una volta come si scrive sul libro e una volta come si scrive nei vari processi di ricerca. Una risposta a una domanda del tipo 'quali libri di Cechov possiede la nostra biblioteca?' dovrebbe anche informarci sui titoli non in inglese.

Sottili discrepanze tra la nozione dell'analista di termini e concetti e il loro significato proprio all'interno del dominio modellato possono avere effetti profondi. Tali disallineamenti possono verificarsi più facilmente in domini che già "conosciamo", come una libreria. Una discussione illuminante sui potenziali problemi nella specifica (formalmente) dei requisiti di un sistema bibliotecario può essere trovata in (Wing, **1988**). I problemi rilevati includono:

- Un impiegato della biblioteca può anche essere un membro della biblioteca; quindi, i due gruppi di utenti del sistema non sono disgiunti.
- C'è una differenza tra un libro (identificato dal suo ISBN) e le copie (fisiche) di un libro di proprietà della biblioteca.
- Non è sufficiente denotare semplicemente lo stato di un libro con un valore booleano (presente o non presente, cioè prestato). Ad esempio, un libro o, più propriamente, una copia di un libro, può essere perso, rubato o in riparazione.

Le persone coinvolte in un UoD hanno un **modello concettuale implicito** di quell'UoD. Un modello concettuale implicito è costituito dalla conoscenza di base condivisa dalle persone nell'UoD. Il fatto che questa conoscenza sia condivisa dà origine a dichiarazioni "ovviamente" da parte di persone all'interno dell'UoD, perché questa conoscenza è data per scontata. ("Naturalmente, una copia di un libro non è la stessa cosa di un libro.") Parte del modello concettuale implicito non è verbalizzato. Contiene conoscenza tacita, conoscenza che è abilmente applicata e funziona in background. Infine, un modello concettuale implicito contiene abitudini, costumi, pregiudizi e persino incongruenze.

Durante la modellazione concettuale, un modello concettuale implicito viene trasformato in un modello esplicito. Così facendo, l'analista si trova di fronte a due tipi di problemi: problemi di analisi e problemi di negoziazione. I problemi di analisi derivano dal fatto che parte del modello concettuale implicito non è verbalizzato, che il modello concettuale implicito si evolve nel tempo, che l'utente e l'analista parlano un linguaggio diverso e che il modello concettuale implicito non può essere completamente codificato. I problemi di negoziazione sorgono perché le persone nell'UoD possono contrastare il processo di analisi, perché i modelli concettuali impliciti delle persone nell'UoD possono differire, oa causa di interessi opposti delle persone coinvolte (come il personale della biblioteca rispetto ai loro dirigenti). Entrambi i tipi di problemi sono discussi di seguito.

Il problema che deve essere affrontato dal sistema automatizzato nasce dall'utente, un essere umano. Questa persona deve essere in grado di descrivere il problema in modo sia corretto che completo. Deve essere comunicato a una persona che in generale ha un background piuttosto diverso. L'analista spesso manca di una conoscenza sufficientemente profonda del dominio applicativo in cui il problema ha avuto origine. Deve imparare la lingua del dominio dell'applicazione e conoscerne la terminologia, i concetti e le procedure. Soprattutto nei progetti di grandi dimensioni, le conoscenze applicative tendono a essere scarsamente diffuse tra gli specialisti coinvolti, il che porta facilmente a difficoltà di integrazione e coordinamento.

In molti casi, ha senso distinguere tra proprietà che denotano una verità oggettiva sul dominio modellato e proprietà che si vorrebbe mantenere nel sistema da sviluppare. Jackson (**2001**) usa i termini **indicativi** per le proprietà del dominio e **ottativi** per i requisiti dell'utente. Le proprietà indicative devono semplicemente essere trattate. Le proprietà ottative sono oggetto di dibattito e negoziazione. L'esempio della libreria mostra già che la distinzione tra questi due tipi di proprietà non è sempre facile da fare.

Nel nostro esempio precedente, il bibliotecario deve esprimere i suoi desideri. È possibile che l'inclusione di due nomi di autori ('Tsjechow' e 'Chekhov') sia vista come un dettaglio ovvio che non deve essere portato avanti esplicitamente. L'analista dall'altra parte del tavolo può ancora avere l'impressione di avere un quadro completo del sistema. Questo tipo di omissione può avere gravi conseguenze.

- Alcuni anni fa, negli Stati Uniti veniva sviluppato un grande sistema di difesa aerea automatizzato. Durante uno dei test finali di questo sistema è stato emesso un segnale di allarme. Uno dei computer ha rilevato un missile sconosciuto. Si è rivelata essere la luna. Questa possibilità non era stata pensata.

Ottenere informazioni corrette e complete è un prerequisito importante per il successo. Questo risulta essere piuttosto problematico nella pratica. Chiedere al potenziale utente cosa si desidera generalmente non funziona. Il più delle volte otteniamo un quadro piuttosto incompleto e impreciso della situazione. Motivi importanti di ciò sono le limitazioni umane nell'elaborazione delle informazioni, nella selezione delle informazioni e nella risoluzione dei problemi. Queste limitate capacità umane sono aggravate da fattori quali:

- la complessità e la variazione dei requisiti che possono essere imposti al software;
- le differenze di background tra il cliente, o utente, e lo specialista del software.

Nella ricerca sull'elaborazione delle informazioni umane, si utilizza spesso un modello in cui la memoria umana è composta da due componenti: la memoria a breve termine in cui vengono elaborate le informazioni e la memoria a lungo termine in cui è archiviata la conoscenza permanente. La memoria a breve termine ha una capacità limitata: si dice spesso che abbia circa sette slot. La memoria a lungo termine, invece, ha una capacità molto grande. Quindi, le informazioni vengono elaborate in una parte relativamente piccola della memoria umana. Si accede quindi alla memoria a lungo termine in modo indiretto. Inoltre, gli esseri umani utilizzano anche memorie esterne quando vengono elaborate informazioni: una lavagna, un pezzo di carta, ecc.

Se una persona intervistata durante l'ingegneria dei requisiti utilizza solo la sua memoria a breve termine, i suoi limiti possono avere un impatto sui risultati. Ciò può verificarsi facilmente se non si fa uso di memorie esterne. Le cose possono essere dimenticate, semplicemente perché la nostra memoria a breve termine ha una capacità limitata.

Gli esseri umani sono anche inclini a essere prevenuti sulla selezione e l'utilizzo delle informazioni. Siamo, in particolare, propensi a far prevalere gli eventi recenti. Nell'elaborare una specifica dei requisiti, ciò porta a requisiti che riguardano la situazione attuale, le informazioni attualmente disponibili, gli eventi recenti, ecc.

Gli esseri umani non sono molto capaci di pensiero razionale. Semplificheranno le cose e useranno un modello che non si adatta davvero alla realtà. Altre limitazioni che influenzano il nostro modello di realtà sono determinate da fattori come l'istruzione, il pregiudizio, la pratica, ecc. Questo stesso tipo di semplificazione si verifica quando vengono elaborati i requisiti del software e il risultato sarà limitato dagli stessi fattori.

Non possiamo sempre aspettarci che l'utente sia in grado di dichiarare con precisione le sue esigenze in una fase iniziale. Un motivo per indagare sulle opportunità dell'automazione è spesso dovuto a una certa insoddisfazione per la situazione attuale. Le persone non sono soddisfatte della situazione attuale e hanno l'impressione che l'automazione aiuterà. Che

questo sia vero o meno – molti problemi di elaborazione dei dati sono problemi organizzativi – la semplice automatizzazione della situazione attuale non è sempre la soluzione. Si vuole qualcosa di diverso, anche se non è chiaro cosa. Solo quando si ottengono informazioni sulle possibilità dell'automazione, si manifesteranno i requisiti reali. Questo è uno dei motivi dell'entità del problema di manutenzione. Circa la metà dello sforzo di manutenzione riguarda l'adattamento del software ai (nuovi) requisiti dell'utente. Per contrastare questa tendenza, i modelli di processo di sviluppo software che riconoscono questo processo di apprendimento, come la prototipazione, lo sviluppo incrementale ei metodi agili, sono da preferire a quelli che non lo fanno, ovvero il modello a cascata e le sue varianti. La forma più estrema è non avere affatto una fase di ingegneria dei requisiti, ma raccogliere ciò che emerge dallo sforzo collaborativo di un folto gruppo di persone, come accade in Wikipedia. Questo è discusso nella **Sezione 9.1.6**.

Attraverso un'analisi attenta, possiamo sperare di costruire una solida prospettiva delle esigenze degli utenti e di anticipare i cambiamenti futuri. Tuttavia, indipendentemente da quanto tempo viene speso in un dialogo con i potenziali utenti, i cambiamenti futuri rimangono difficili da prevedere. Potremmo fare un ulteriore passo avanti e stabilire che i requisiti non saranno *mai* completi. A questo proposito, specificare i requisiti ha molto in comune con la previsione del tempo: c'è un limite a quanto lontano si può prevedere il futuro.

In una situazione in cui l'obiettivo di un progetto di sviluppo software è migliorare un "sistema" esistente, sia esso un processo manuale o uno (parzialmente) automatizzato, è generalmente utile distinguere esplicitamente due fasi di modellazione. Nella prima fase, viene modellata la situazione attuale. Sulla base di un'analisi dei punti di forza e di debolezza della situazione attuale, viene quindi modellata la situazione futura. La riprogettazione dei processi aziendali (BPR), in particolare, sottolinea la distinzione tra queste due fasi di modellazione.

Perché la fase di ingegneria dei requisiti abbia successo, abbiamo bisogno di metodi e tecniche che cerchino di aggirare le difficoltà sopra delineate. Il grado in cui sono richieste tecniche potenti dipende dall'esperienza delle persone coinvolte nella fase di ingegneria dei requisiti (sia utenti che analisti) e dall'esperienza dell'analista con il dominio dell'applicazione. **La sezione 9.1.2** discute una serie di tecniche per l'elicitazione dei requisiti.

Ma prima di discutere queste tecniche, elaboriamo innanzitutto nella **Sezione 9.1.1** come diverse visioni del mondo si traducano in approcci diversi all'ingegneria dei requisiti. Nella **Sezione 9.1.3** , discutiamo di come i requisiti si collegano a obiettivi di livello superiore e di come punti di vista diversi possono comportare insiemi di requisiti diversi e talvolta contrastanti. Nelle **Sezioni 9.1.4** e **9.1.5** , discutiamo come dare la priorità ai requisiti e come i requisiti sono correlati alla selezione dei componenti COTS.

9.1.1 Paradigmi ingegneristici dei requisiti

La maggior parte dei metodi di ingegneria dei requisiti e dei metodi di sviluppo software in generale sono di natura tayloriana. Intorno alla fine del 20° secolo, Taylor ha introdotto la nozione di "gestione scientifica", in cui i compiti sono scomposti ricorsivamente in compiti più semplici e ogni compito ha un "modo migliore" per portarlo a termine. Attraverso attente osservazioni ed esperimenti questo modo migliore può essere trovato e formalizzato in procedure e regole. La gestione scientifica è stata applicata con successo in molte operazioni di fabbrica. L'equivalente nell'ingegneria dei requisiti è intervistare esperti di dominio e osservare gli utenti finali al lavoro per ottenere i requisiti degli utenti "reali". Successivamente, gli esperti si mettono al lavoro e implementano questi requisiti. Durante quest'ultimo processo, non è più necessario interagire con la comunità degli utenti. Questa visione dello sviluppo del software è funzionale e razionale. Il presupposto di fondo è che esiste una verità oggettiva, che deve semplicemente essere scoperta durante il processo di analisi.

Sebbene questa visione abbia i suoi meriti nell'elaborare requisiti in ambiti puramente *tecnici,* molti universi del discorso coinvolgono anche persone, persone il cui modello del mondo è incompleto, soggettivo, irrazionale e può entrare in conflitto con la visione del mondo degli altri. In questi casi, l'analista non è un osservatore esterno passivo dell'UoD. Piuttosto, partecipa attivamente alla *formazione* dell'UoD.

Viene sempre più riconosciuto che l'approccio tayloriano, funzionale, non è l'unico, e non deve essere necessariamente il più appropriato, approccio al processo di ingegneria dei requisiti.

Gli analisti hanno una serie di ipotesi sulla natura dell'argomento di studio. Tale insieme di ipotesi è comunemente chiamato "paradigma". Nel nostro campo, questi presupposti riguardano il modo in cui gli analisti acquisiscono conoscenza (assunti epistemologici) e la loro visione del mondo sociale e tecnico (assunti ontologici).

I presupposti sulla conoscenza sfociano in una dimensione oggettivista-soggettivista. Se l'analista assume il punto di vista oggettivista, applica modelli e metodi derivati dalle scienze naturali per arrivare all'unica verità. Nella posizione soggettivista, la principale preoccupazione dell'analista è capire come l'individuo crea, modifica e interpreta il mondo in cui si trova.

Le ipotesi sul mondo si traducono in una dimensione di ordine-conflitto. Il punto di vista dell'ordine enfatizza l'ordine, la stabilità, l'integrazione e il consenso. D'altra parte, la visione del conflitto sottolinea il cambiamento, il conflitto e la disintegrazione. Queste due dimensioni e le posizioni estreme ad esse associate producono quattro paradigmi per l'ingegneria dei requisiti e, più in generale, per lo sviluppo dei sistemi informativi:

- **Funzionalismo** (obiettivo-ordine). Nel paradigma funzionalista, lo sviluppatore è l'esperto di sistema che ricerca relazioni di causa-effetto misurabili. Si ritiene che esista una realtà

organizzativa empirica, indipendente dall'osservatore. I sistemi sono sviluppati per supportare il funzionamento organizzativo razionale. La loro efficacia ed efficienza può essere verificata oggettivamente, con prove simili a quelle utilizzate in altre discipline ingegneristiche.

- **Social-relativismo** (ordine soggettivo). In questo paradigma, l'analista opera come facilitatore. La realtà non è qualcosa di immutabile "là fuori", ma è costruita nella mente umana. L'analista è un agente di cambiamento. Cerca di facilitare l'apprendimento di tutte le persone coinvolte.
- **Strutturalismo radicale** (obiettivo-conflitto). Nel paradigma radicale l'assunto chiave è che lo sviluppo del sistema intervenga nel conflitto tra due o più classi sociali per potere, prestigio e risorse. I sistemi sono spesso sviluppati per sostenere gli interessi dei proprietari, a scapito degli interessi della forza lavoro. Per riequilibrare l'equilibrio di potere, questo paradigma suggerisce che l'analista dovrebbe agire come un partigiano del lavoro. I requisiti di sistema dovrebbero evolvere da una cooperazione tra la forza lavoro e l'analista. Si ritiene che questo approccio porti a sistemi che migliorano l'artigianato e le condizioni di lavoro.
- **Neoumanesimo** (soggettivo-conflitto). Il tema centrale di questo paradigma è l'emancipazione. I sistemi sono sviluppati per rimuovere le influenze distorsive e altre barriere al discorso razionale. Lo sviluppatore del sistema agisce come un terapista sociale nel tentativo di riunire, in una discussione aperta, un gruppo eterogeneo di individui, inclusi i clienti, la forza lavoro e vari livelli di gestione.

È vero che questi paradigmi riflettono orientamenti estremi. In pratica, una combinazione di presupposti di solito guiderà il processo di ingegneria dei requisiti. Eppure è giusto dire che la maggior parte delle tecniche di sviluppo del sistema enfatizzano la visione funzionalista.

Nella dimensione soggettivista-oggettivista, è importante rendersi conto che una buona parte del soggettivismo può essere coinvolta nella formazione dell'UoD. Se dobbiamo sviluppare un sistema per controllare una fotocopiatrice, possiamo tranquillamente prendere una posizione funzionale. Possiamo aspettarci che una macchina del genere funzioni in modo puramente razionale. Nel processo di analisi, elenchiamo le funzioni della macchina, i suoi segnali interni, le condizioni e così via, al fine di ottenere un quadro soddisfacente del sistema da sviluppare. Una volta identificati questi requisiti, possono essere congelati e un modello di processo simile a una cascata può essere utilizzato per realizzare il sistema. Se, invece, il nostro compito è quello di sviluppare un sistema di supporto alle persone nello svolgimento del proprio lavoro, come un sistema di office automation, una visione del mondo puramente funzionale può facilmente portare a sistemi mal concepiti. In tali casi, la partecipazione dell'utente finale alla definizione dell'UoD è di fondamentale importanza. Attraverso un dialogo aperto con le persone interessate, possiamo incoraggiare i potenziali utenti a influenzare il sistema da sviluppare. Parte del lavoro dell'analista in questo caso è riconciliare le opinioni dei partecipanti al processo di analisi. Un feedback continuo durante le fasi di costruzione effettive con possibilità di reindirizzamento può aumentare ulteriormente le possibilità di

successo. Sono gli utenti futuri che lavoreranno con il sistema. A nulla serve confrontarli con un sistema che non soddisfa i loro bisogni.

L'automazione trasforma le organizzazioni e, quindi, influisce sui dipendenti dell'organizzazione. Può suscitare paure e altre emozioni nei dipendenti interessati. Ad esempio, il nostro sistema bibliotecario offre potenzialmente alle persone l'accesso a molte più informazioni di quelle che avevano in precedenza. Alcune persone potrebbero preferire avere accesso solo alle informazioni relative ai loro compiti e responsabilità. Durante l'ingegneria dei requisiti, dobbiamo essere consapevoli di questi effetti. Il paradigma puramente funzionalista quindi non serve a nulla.

Un utente insoddisfatto cercherà di trascurare il sistema o, nella migliore delle ipotesi, di esprimere immediatamente ulteriori requisiti. Il risultato netto è che il previsto guadagno in termini di efficienza o efficacia non viene raccolto.

Un esempio illuminante e ben documentato dei possibili effetti del seguire un paradigma abbastanza radicale è fornito in (Page *et al.*, **1993**); vedere anche **la Sezione 1.4.3** . Sebbene il sistema di spedizione computerizzato per il London Ambulance Service avrebbe avuto un impatto significativo sul modo in cui gli equipaggi delle ambulanze svolgevano il loro lavoro, c'era poca consultazione con loro. Alcune delle conseguenze di questo approccio sono state le seguenti (Page *et al.*, **1993** , pp. 40–41):

- Il sistema allocava la risorsa disponibile più vicina indipendentemente dalla stazione di origine, quindi gli equipaggi spesso dovevano operare sempre più lontano dalla loro base. Ciò li ha portati a operare in un territorio sconosciuto con ulteriori spostamenti per raggiungere la loro stazione di origine alla fine di un turno.
- Il nuovo sistema ha tolto la flessibilità che gli equipaggi avevano in precedenza per la stazione di decidere quale risorsa allocare. Ciò comportava inevitabilmente problemi quando veniva utilizzata una risorsa diversa da quella assegnata.
- La mancanza di contatto vocale ha reso l'intero processo più impersonale e ha esacerbato la situazione "loro e noi".

Se i modelli concettuali dei partecipanti differiscono, possiamo cercare un compromesso o optare per una delle opinioni espresse. È impossibile fornire linee guida generali su come gestire tali casi. Cercare un compromesso può essere un affare noioso e può portare a un sistema di cui nessuno è veramente soddisfatto. Optare per una particolare visione del mondo renderà felice una parte, ma potrebbe far sì che altri trascurino completamente il sistema sviluppato. Peggio ancora, potrebbero decidere di sviluppare un sistema concorrente.

9.1.2 Tecniche di Elicitazione dei Requisiti

Le due principali fonti di informazione per il processo di elicitazione dei requisiti sono gli utenti e il dominio (dell'applicazione). Queste fonti presuppongono entrambe che esista qualcosa "là fuori" con cui iniziare, da cui possono essere elicitati i requisiti. Nello sviluppo di software

orientato al mercato, tuttavia, spesso non è così; l'elicitazione dei requisiti in tali progetti è più simile all'invenzione dei requisiti o alla formulazione di problemi, guidata da considerazioni di marketing e vendita.

La Figura 9.3 elenca una serie di tecniche di elicitazione, che vengono elaborate di seguito. La figura ci dice anche che l'utente è la principale fonte di informazioni in alcune tecniche, mentre il dominio è predominante in altre. Inoltre, la figura indica se ciascuna tecnica è particolarmente utile per modellare la situazione attuale o quella futura anticipata.

In genere dovresti aspirare un tappeto in due direzioni anziché in una; allo stesso modo, dovresti usare più tecniche di elicitazione dei requisiti.

Technique	Main info source		Strong on	
	Domain	User	Current	Future
Interview		X	X	
Delphi technique		X	X	
Brainstorming session		X		X
Task analysis		X	X	
Scenario (use-case) analysis		X	X	X
Ethnography	X		X	
Form analysis	X		X	
Analysis of natural language descriptions	X		X	
Synthesis of reqs from an existing system	X		X	
Domain analysis	X		X	
Use of reference models	X		X	
Business process redesign (BPR)	X		X	X
Prototyping		X		X

Figure 9.3 Tecniche di elicitazione dei requisiti

Intervistare/Chiedere: potremmo semplicemente chiedere agli utenti cosa si aspettano dal sistema. Un presupposto quindi è che l'utente sia in grado di aggirare i propri limiti e pregiudizi. La domanda può assumere la forma di un'intervista, un brainstorming o un questionario. In un'intervista aperta, l'utente parla liberamente dei suoi compiti. Questa è la forma più semplice di elicitazione dei requisiti, ma presenta tutti gli inconvenienti menzionati prima. In un colloquio strutturato, l'analista cerca di superarli guidando l'utente, ad esempio attraverso domande chiuse o probanti.

Nelle sessioni di discussione con un gruppo di utenti, scopriamo spesso che alcuni utenti sono molto più articolati di altri e quindi hanno una maggiore influenza sul risultato. Il consenso così raggiunto non deve essere equilibrato. Per superare questo problema, può essere impiegata una tecnica Delphi. La **tecnica Delphi** è una tecnica iterativa in cui le informazioni vengono scambiate in forma scritta fino al raggiungimento di un consenso. Ad esempio, i partecipanti possono annotare i propri requisiti, ordinati in ordine di importanza. I set di requisiti così ottenuti vengono distribuiti a tutti i partecipanti, che riflettono su di essi per

ottenere un set di requisiti rivisto. Questa procedura viene ripetuta più volte fino a raggiungere un consenso sufficiente.

Per i prodotti di consumo, come pacchetti di elaborazione testi, software antivirus o software per l'amministrazione personale, gli utenti hanno spesso la possibilità di fornire feedback, porre domande, segnalare bug e così via, elettronicamente. Questo tipo di informazioni viene inoltre regolarmente raccolto e conservato dagli addetti alle vendite e al marketing nel corso dei loro contatti con i clienti. Questi registri possono essere estratti e in questo modo forniscono una preziosa fonte di informazioni quando si cercano i requisiti per la prossima versione di quel software.

Analisi delle attività: i dipendenti che lavorano in un determinato dominio svolgono una serie di attività, come la gestione delle richieste di prestito di un libro, la catalogazione di nuovi libri, l'ordinazione di libri, ecc. Le attività di livello superiore possono essere scomposte in sottoattività. Ad esempio, l'attività "Gestisci la richiesta di prendere in prestito un libro" può portare alle seguenti attività secondarie:

- controllare l'identificazione dei membri,
- verificare il limite al numero di libri che possono essere presi in prestito,
- registrare il libro come preso in prestito dal membro della biblioteca,
- emettere una ricevuta indicando la data di scadenza.

L'analisi delle attività è una tecnica per ottenere una gerarchia di attività e attività secondarie che devono essere eseguite dalle persone che lavorano nel dominio. Qualsiasi delle altre tecniche discusse può essere utilizzata per ottenere le informazioni necessarie per tracciare questa gerarchia. Non ci sono regole chiare su quando interrompere le attività di scomposizione. Una delle principali euristiche è che a un certo punto gli utenti tendono a "rifiutarsi" di scomporre ulteriormente le attività. Ad esempio, quando viene chiesto come viene verificata l'identificazione del membro, l'impiegato della biblioteca può dire "Bene, controllo semplicemente il suo documento d'identità". A questo punto, un'ulteriore scomposizione non ha senso.

L'analisi dei compiti viene spesso applicata nella fase in cui (dettagli sulla) componente di interazione uomo-computer vengono decisi. Questo sottovaluta la sua potenza come tecnica di elicitazione dei requisiti generali. Dà anche l'impressione (sbagliata) che gli utenti siano interessati solo all'aspetto grafico dell'interfaccia.

Analisi basata sullo scenario: invece di cercare piani generici, come nelle interviste o nell'analisi delle attività, l'analista può studiare le **istanze** delle attività. Uno scenario è una storia che ci racconta come viene eseguita un'istanza di attività specifica. Lo scenario può essere reale o artificiale. Un esempio di uno scenario reale è che l'analista osserva come un impiegato della biblioteca gestisce una richiesta dell'utente effettiva. Possiamo chiedere al dipendente della biblioteca di verbalizzare ciò che sta facendo e di fare una registrazione audio o video. Questo metodo **di riflessione ad alta voce** è una tecnica abbastanza discreta

per studiare le persone al lavoro. Viene spesso utilizzato per valutare prototipi o sistemi informativi esistenti.

In alternativa, possiamo costruire scenari artificiali e discuterli con l'utente. Come prima cosa, possiamo ad esempio elaborare il seguente scenario per la restituzione di un libro:

1. La data di scadenza del libro è verificata. Se il libro è scaduto, il socio è tenuto al pagamento della relativa multa.
2. Il libro è registrato come idoneo per il check-out.
3. Il libro viene rimesso al suo posto.

Quando questo scenario viene discusso con il dipendente della biblioteca, possono emergere una serie di problemi correlati, sia attraverso domande di indagine dell'analista o perché l'utente contrasta lo scenario con la pratica quotidiana. Esempi di domande che potrebbero essere poste includono cose come:

- Cosa succede se la persona che restituisce il libro non è un membro registrato della biblioteca?
- Cosa succede se il libro restituito è danneggiato?
- Cosa succede se il membro che restituisce questo libro ha altri libri scaduti o una prenotazione in sospeso per un altro libro?

In sostanza, questo tipo di narrazione fornisce all'utente una versione simulata artificiale del software che verrà eventualmente consegnata. Serve come prototipo cartaceo per comprendere meglio i requisiti. Se legata alla modellazione di tipo UML, l'analisi basata su scenari è spesso chiamata **analisi dei casi d'uso**; vedere la Sezione 10.3.6 . Gli scenari e i casi d'uso sono i metodi di elicitazione più utilizzati.

L'analisi basata sullo scenario viene spesso eseguita in modo alquanto casuale. In tal caso, non è possibile stabilire se siano stati elaborati scenari sufficienti e se si sia ottenuto un quadro sufficientemente accurato e completo dei requisiti. Scrivere buoni scenari non è affatto facile. Sebbene possa sembrare banale "registrare semplicemente gli episodi degli utenti", è necessaria una discreta esperienza di dominio per ottenere una serie di scenari validi e affidabili.

Gli scenari possono essere visti da diverse prospettive. Nello scenario di esempio precedente per la restituzione di un libro, lo scenario elenca una serie di azioni o eventi che insieme costituiscono un episodio. L'attenzione è quindi sull'aspetto del processo, che mostra come il sistema procede attraverso stati successivi. In alternativa, lo stesso scenario può essere considerato dal punto di vista dell'utente: come interagisce l'utente con il sistema? Quali funzionalità le vengono offerte? Ancora un'altra prospettiva è che lo scenario porta a discussioni su alternative dalle quali deve essere fatta una certa scelta, come nelle domande sollevate dallo scenario di esempio sopra.

Etnografia: uno dei principali svantaggi dell'elicitazione dei requisiti attraverso, ad esempio, le interviste è che l'analista impone all'utente la sua visione di come è ordinato il mondo. Tali metodi potrebbero non riuscire se l'analista e l'utente non condividono un sistema di categorie. L'analista può, ad esempio, porre la seguente domanda: "Se un membro vuole prendere in prestito un libro ma ha una multa in sospeso, lei:

1. Rifiuta la richiesta, o
2. Gestisci la richiesta comunque.'

Questa scelta binaria non ha bisogno di mappare la pratica effettiva. Il dipendente della biblioteca può, ad esempio, accogliere la richiesta purché sia saldata parte della multa arretrata o se sa che il membro è degno di fiducia.

L'**etnografia** (dal greco: *ethnos (ἔθνος)* - "popolo", e *grapho (γράφω)* - "scrivere"; letteralmente "descrizione del popolo") è il metodo con cui operano le ricerche sul campo delle scienze etnoantropologiche. Fare etnografia significa recarsi tra coloro che si vuole studiare per un certo periodo di tempo, ed utilizzare alcune tecniche di ricerca (come l'osservazione o l'intervista) allo scopo di collezionare un insieme di dati che una volta interpretati, rendano possibile la comprensione della cultura in esame. Riti, rituali, cerimonie, norme, valori, credenze, comportamenti, artefatti, sono i principali fenomeni di interesse dell'etnografo, attraverso i quali la cultura si rende intelligibile.

I protocolli di pensiero ad alta voce si basano sull'idea che gli utenti hanno obiettivi e sotto-obiettivi ben definiti e che attraversano tali alberi degli obiettivi in modo ordinato dall'alto verso il basso. Le persone però spesso non hanno piani precostituiti, ma procedono piuttosto in modi un po' opportunistici.

Uno svantaggio dell'analisi dei compiti è che considera i compiti individuali delle singole persone, senza tener conto dell'ambiente sociale e organizzativo in cui questi compiti vengono eseguiti.

Si sostiene che i metodi etnografici non presentino tali carenze. In etnografia, i gruppi di persone vengono studiati nel loro ambiente naturale. È noto dalla sociologia dove, ad esempio, le tribù polinesiane vengono studiate vivendo con loro per un lungo periodo di tempo. Allo stesso modo, le esigenze degli utenti possono essere studiate partecipando al loro lavoro quotidiano per un periodo di tempo, ad esempio diventando un impiegato di biblioteca. L'analista diventa apprendista, riconoscendo che i futuri utenti del sistema sono i veri esperti del loro lavoro. I metodi etnografici hanno maggiori probabilità di scoprire la conoscenza tacita rispetto alla maggior parte delle altre tecniche di elicitazione.

Analisi dei moduli: molte informazioni sul dominio modellato possono essere spesso trovate in vari moduli utilizzati. Ad esempio, per richiedere alcuni atti di una conferenza da un'altra libreria, l'utente potrebbe dover compilare un modulo come quello della **Figura 9.4** .

I moduli ci forniscono informazioni sugli oggetti dati del dominio, le loro proprietà e le loro interrelazioni. Sono particolarmente utili come input per modellare l'aspetto dei dati del sistema; vedere anche **la Sezione 10.1.1** .

Proceedings Request Form	
Member name	.....
Member address	.....
	.....
Title	.....
Series no	.....
Editor	.....
Place	.....
Publisher	.....
Year	.....
Signature	.....

Figure 9.4 Un modulo di esempio

Gli utenti delle biblioteche spesso hanno una conoscenza incompleta delle fonti di informazione a cui sono interessati. Ad esempio, qualcuno potrebbe cercare gli atti della *Conferenza internazionale sull'ingegneria del software* che si è svolta a Berlino. Solo se le varie voci del modulo precedente vengono utilizzate come entità nel modello di dati sottostante è possibile rispondere facilmente a tale query. In questo caso, il modulo punta direttamente a un'esigenza utile che altrimenti potrebbe passare inosservata.

Descrizioni in linguaggio naturale Come i moduli, le descrizioni in linguaggio naturale forniscono molte informazioni utili sul dominio da modellare. Le istruzioni per l'uso per i dipendenti della biblioteca potrebbero ad esempio contenere un paragrafo come quello riportato nella **Figura 9.5**. Questo testo ci fornisce informazioni come:

- Ci sono (almeno) due conti su cui gli ordini possono essere addebitati.
- C'è un elenco di membri del personale autorizzati ad approvare tali richieste.

- Esiste la possibilità di ordinare più copie di un titolo per conto degli studenti.

Spesso, le descrizioni (e le forme) del linguaggio naturale forniscono all'analista informazioni di base da utilizzare insieme ad altre tecniche di elicitazione come le interviste. Le descrizioni in linguaggio naturale, in particolare, tendono a presupporre molta tacita conoscenza da parte del lettore. Ad esempio, se il modulo B contiene un ISBN, ciò consente di risparmiare un po' di lavoro al dipendente della biblioteca, ma probabilmente la richiesta verrà comunque gestita se queste informazioni non vengono fornite. Un problema pratico con le descrizioni in linguaggio naturale è che spesso non vengono mantenute aggiornate. Come la

Figure 9.5 Un esempio di istruzione per i dipendenti della biblioteca

> *Prima che una richiesta di acquisizione di un libro possa essere soddisfatta, il modulo B deve essere compilato completamente. Una richiesta non può essere gestita se non è firmata da un membro del personale autorizzato o se non è indicato il conto da addebitare (Studente' o 'Staff). La richiesta non può essere accolta se il titolo richiesto è già presente nel catalogo dei titoli, a meno che non sia contrassegnato come Rubato o 'Smarrito', o il conto sia 'Studente'.*

documentazione del software, la loro validità tende a deteriorarsi nel tempo.

Derivazione da un sistema esistente: Partendo da un sistema esistente, ad esempio un sistema simile in qualche altra organizzazione o una descrizione in un libro di testo, possiamo formulare i requisiti del nuovo sistema. Ovviamente, dobbiamo stare attenti e tenere conto delle circostanze peculiari della situazione attuale.

Piuttosto che guardare un sistema particolare, possiamo anche studiare un certo numero di sistemi in qualche dominio applicativo. Questo processo di analisi dei metarequisiti è noto come **analisi del dominio.** Il suo obiettivo generalmente è identificare componenti, concetti, strutture riutilizzabili e così via. È pericoloso cercare requisiti riutilizzabili in domini immaturi. I requisiti possono quindi essere riutilizzati semplicemente perché sono disponibili, non perché si adattano alla situazione attuale. Diventano 'legno morto'. Nel contesto dell'analisi dei requisiti, l'analisi del dominio può essere vista come una tecnica per derivare un modello di "riferimento" per i sistemi all'interno di un determinato dominio. Tale modello di riferimento fornisce uno scheletro (architettura) che può essere ampliato e adattato per adattarsi alla situazione specifica a portata di mano.

L'analisi del dominio è ulteriormente discussa nel **Capitolo 17** , nel contesto del riutilizzo del software.

Riprogettazione dei processi aziendali (BPR): in molti progetti di sviluppo software, le persone coinvolte saltano rapidamente a conclusioni: l'automazione è la risposta. Ancora

peggio, la loro conclusione potrebbe essere che automatizzare la situazione attuale è la risposta. Nella riprogettazione dei processi aziendali (o reingegnerizzazione dei processi aziendali), viene seguita una strategia piuttosto diversa. Si tratta di un'attività organizzativa per riprogettare radicalmente i processi aziendali per ottenere risultati competitivi in termini di qualità, costi o soddisfazione degli utenti. In BPR, ci allontaniamo completamente dai modi esistenti di fare le cose. BPR prevede i seguenti passaggi:

1. Identificare i processi per l'innovazione. Due approcci principali per farlo sono l'approccio esaustivo e ad alto impatto. Nell'approccio esaustivo, si tenta di identificare tutti i processi, a cui viene quindi data la priorità per la loro urgenza di riprogettazione. L'approccio ad alto impatto cerca di identificare solo i processi più importanti o quelli in conflitto con la visione aziendale.
2. Identificare le leve di cambio. In questa fase vengono identificate le opportunità per facilitare il miglioramento dei processi. Si possono riconoscere tre tipi di leve: fattori abilitanti organizzativi (come responsabilizzare i team), fattori abilitanti delle risorse umane (come l'arricchimento delle attività) e abilitanti della tecnologia dell'informazione.
3. Sviluppare una visione del processo. Affinché la riprogettazione abbia successo, l'organizzazione deve sapere quali obiettivi vuole raggiungere. Questo è descritto nella visione del processo. Le componenti principali di una visione di processo sono: obiettivi di processo (target misurabili delle prestazioni future del sistema), attributi di processo (proprietà qualitative e descrittive del processo futuro), fattori critici di successo e vincoli (organizzativi, culturali e tecnologici).
4. Comprendi il processo esistente. Ciò include la documentazione del processo esistente, la sua misurazione e l'identificazione degli aspetti problematici. Ci consente di valutare lo stato di salute del processo esistente e fa emergere problemi.
5. Progetta e prototipa il nuovo processo. Questo è il passo finale. La prototipazione consente di sperimentare nuove strutture, riducendo così il rischio di fallimento.

BPR non è in realtà una tecnica di elicitazione dei requisiti vera e propria. Viene qui menzionato perché sottolinea una questione essenziale da affrontare durante la fase di ingegneria dei requisiti. I processi aziendali non dovrebbero essere guidati dalla tecnologia dell'informazione. Piuttosto, la tecnologia dell'informazione dovrebbe consentirli. Sebbene uno sforzo completo di BPR non sia necessario o fattibile in molte situazioni, ripensare i processi e le procedure esistenti è un passaggio che troppo spesso viene ignorato in modo sconsiderato nei progetti di sviluppo software.

Ad esempio, consideriamo ancora una volta il nostro progetto di automazione delle biblioteche. Un'attenta ispezione della situazione attuale potrebbe rivelare che le cose non sono poi così male. Tuttavia, l'impressione è che il numero di richieste che non è stato possibile esaudire sia cresciuto costantemente negli ultimi anni. Questa è percepita come la causa principale del crescente numero di utenti insoddisfatti. Poiché il servizio ai propri clienti ha un'elevata priorità, uno degli obiettivi è ridurre del 50% il numero di richieste che non possono essere soddisfatte entro due anni. Affinché ciò sia possibile, la biblioteca dovrebbe

essere autorizzata a spendere il budget disponibile a propria discrezione, piuttosto che essere attivata solo dai segnali dei ricercatori (sembra radicale, non è vero). Si è quindi deciso di potenziare il sistema automatizzato esistente con moduli per tenere traccia delle richieste andate a buon fine e di quelle non andate a buon fine. Sulla base delle informazioni acquisite da questo processo di misurazione durante un periodo di tre mesi, verrà presa una decisione sull'entità della riallocazione di una percentuale del budget annuale.

Prototipazione Dato che è difficile, se non impossibile, costruire il sistema giusto fin dall'inizio, potremmo decidere di utilizzare dei prototipi. Partendo da una prima serie di requisiti, viene costruito un prototipo del sistema. Questo prototipo viene utilizzato per esperimenti, che portano a nuovi requisiti e maggiori informazioni sui possibili usi del sistema. In uno o più passaggi successivi, viene sviluppato un insieme più definito di requisiti. La prototipazione è discussa nella **Sezione 3.2.1** . Altri processi agili seguono una strategia simile in cui i requisiti vengono rapidamente tradotti in un sistema funzionante che deve essere valutato dai suoi utenti.

Di queste tecniche di elicitazione dei requisiti, "chiedere" è la strategia meno certa, mentre "prototipare" è la meno incerta. Oltre all'esperienza degli utenti e degli analisti, l'incertezza del processo è influenzata anche dalla stabilità dell'ambiente, dalla complessità del prodotto da sviluppare e dalla familiarità con l'area problematica in questione. Possiamo provare a stimare l'impatto di tali fattori sulla vulnerabilità della specifica dei requisiti risultante e quindi decidere un metodo principale per l'elicitazione dei requisiti sulla base di questa stima.

Per un problema ben compreso, con analisti molto esperti, può essere sufficiente intervistare i potenziali utenti. Tuttavia, se si tratta di un problema avanzato e mal compreso all'interno di un ambiente in rapido cambiamento e gli analisti hanno poca o nessuna esperienza nel settore in questione, sembra saggio seguire un processo agile.

L'incertezza dei requisiti non è l'unico problema che i project manager devono affrontare e un processo diverso non è l'unica soluzione che scelgono. Gli aspetti politici (come agende nascoste e conflitti tra le parti interessate) sono spesso visti come rischi maggiori della semplice incertezza dei requisiti (Moynihan, **2000**). Naturalmente, questi sono correlati. In entrambi i casi, è molto probabile che i requisiti cambino. I project manager spesso seguono un percorso formale per gestire i disaccordi tra le parti interessate e lasciare che i clienti firmino il documento sui requisiti. Tuttavia, se questa sia la risposta a lungo termine è discutibile.

Man mano che l'incertezza diminuisce, gli effetti benefici della partecipazione degli utenti all'ingegneria dei requisiti diminuiscono. Con maggiore incertezza, tuttavia, una maggiore partecipazione degli utenti ha un effetto positivo sulla qualità dell'ingegneria dei requisiti.

In genere è consigliabile avere più collegamenti cliente-sviluppatore in un progetto di sviluppo software, e in particolare durante l'ingegneria dei requisiti. Keil e Carmel (**1995**) hanno studiato la relazione tra il successo del progetto e il numero e il tipo di tali collegamenti

cliente-sviluppatore. Gli autori hanno osservato una forte correlazione tra il numero di collegamenti e il successo del progetto: più collegamenti implicavano progetti di maggior successo. Il contributo relativo al successo del progetto diminuisce all'aumentare del numero di collegamenti; non è necessario avere più di, diciamo, una mezza dozzina di link. Un'ulteriore osservazione interessante di questo studio è che i collegamenti con utenti *diretti* hanno un impatto maggiore sul successo del progetto rispetto ai collegamenti con utenti *indiretti* come rappresentanti degli utenti o addetti alle vendite. Infine, è stato notato che i progetti di sviluppo orientati al cliente tendono a utilizzare e preferiscono diversi tipi di collegamento ai progetti di sviluppo orientati al mercato. Ad esempio, il collegamento preferito per lo sviluppo personalizzato - i team facilitati - non è stato utilizzato dagli sviluppatori di pacchetti, mentre il collegamento preferito per gli sviluppatori di pacchetti - le linee di supporto - è stato utilizzato raramente per i progetti personalizzati.

Dovremmo essere molto attenti nella nostra valutazione di quale tecnica di elicitazione dei requisiti scegliere. È fin troppo comune essere troppo ottimisti sulla nostra capacità di valutare correttamente i requisiti del software.

Ad esempio, considera il seguente aneddoto di un giornale olandese. Un'azienda nel settore dell'automazione agricola aveva sviluppato un sistema in cui i microchip venivano inseriti nelle orecchie delle mucche. Successivamente, ogni singola vacca poteva essere tracciata: la fornitura di cibo e acqua veniva regolata e regolata, la quantità e la qualità del latte registrate e analizzate automaticamente, ecc. Naturalmente, questa stessa tecnica è stata successivamente applicata con successo ai suini. Successivamente, è stato provato sulle capre. È stato costruito un allevamento di capre completamente automatizzato da un milione di dollari. Ma ahimè, le cose non hanno funzionato così bene per le capre. A differenza delle mucche e dei maiali, le capre mangiano di tutto, comprese le patatine dei loro compagni.

9.1.3 Obiettivi e punti di vista

In questa sezione, discutiamo due modi per **strutturare** un insieme di requisiti. Un modo per farlo è in una struttura gerarchica: i requisiti di livello superiore vengono scomposti in quelli di livello inferiore. I requisiti di alto livello sono spesso definiti obiettivi. L'altro metodo di strutturazione collega i requisiti a parti interessate specifiche. La gestione può avere un insieme di requisiti e gli utenti finali possono avere un altro insieme di requisiti. Questi diversi insiemi di requisiti sono chiamati punti di vista. In entrambi i casi, elicitazione e strutturazione vanno di pari passo.

Ad esempio, uno dei requisiti richiesti per il nostro sistema bibliotecario potrebbe essere che il sistema dovrebbe consentire agli utenti di cercare un determinato libro nel database. Chiedendo a noi stessi o alle parti interessate *perché* questo requisito è necessario, viene rilevato un requisito di livello superiore, ovvero la necessità di disporre di strutture di ricerca. Sempre chiedendosi "perché", si arriva a un obiettivo di alto livello di servire i clienti. Andando dall'altra parte, chiedendosi *come* il sistema bibliotecario possa aiutare a servire i clienti,

potrebbe emergere l'esigenza di conoscere le preferenze dell'utente e di utilizzare questa conoscenza mentre si interagisce con l'utente. In questo modo, ponendosi domande sul *perché* e *sul come,* si sviluppa una struttura gerarchica di obiettivi e requisiti. **La Figura 9.6** contiene un esempio di tale struttura gerarchica.

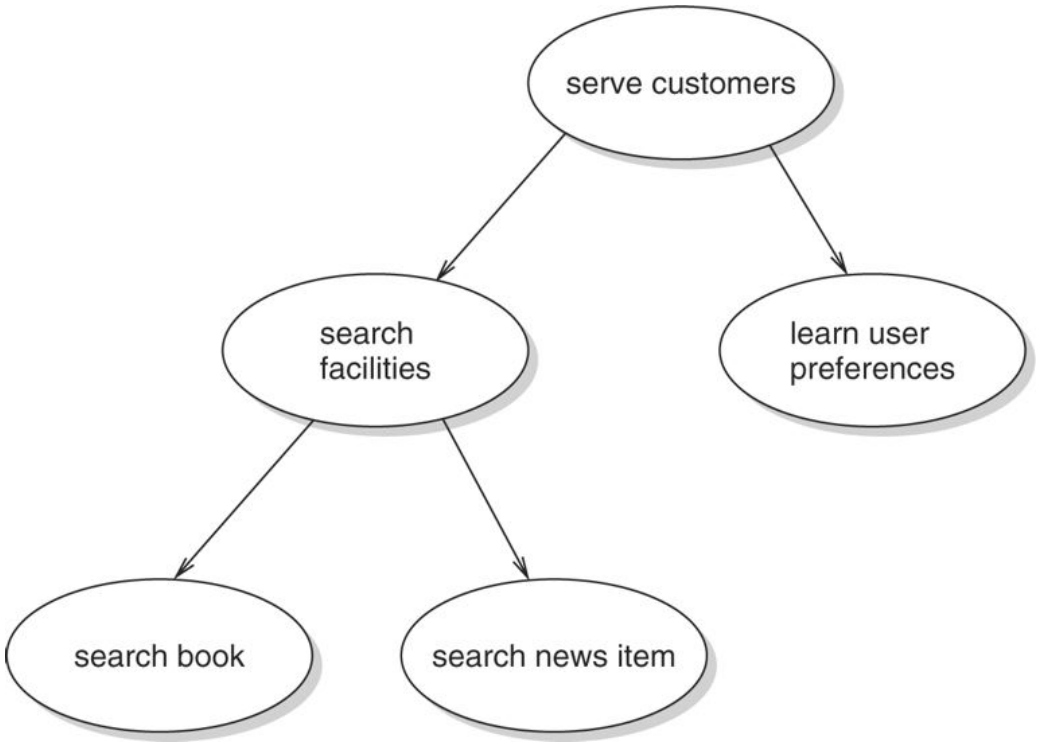


Figure 9.6 Gerarchia dei requisiti

La Figura 9.6 illustra una struttura di raffinamento, in cui ogni requisito viene raffinato (scomposto) in un insieme di sottorequisiti che insieme soddisfano il requisito principale. I sottorequisiti sono correlati a AND: "cerca libro" e "cerca notizia" insieme costituiscono il requisito "strumenti di ricerca".

Possiamo includere anche altri tipi di relazione. Ad esempio, potremmo avere determinate opzioni per un requisito particolare e, a tal fine, avere relazioni OR accanto a relazioni AND. Potremmo anche includere altri tipi di link. Se abbiamo l'obbligo di imporre sanzioni ai clienti che restituiscono gli articoli in ritardo, potremmo concepire ciò come in conflitto con il nostro obiettivo di servire i clienti e collegare questi due requisiti tramite un collegamento di tipo "conflitti con".

Questa cosiddetta **ingegneria dei requisiti basata sugli obiettivi** si traduce in un grafico che collega gli obiettivi di alto livello ai requisiti di livello inferiore. È possibile ragionare su questo grafico, ad esempio per convalidare che determinati obiettivi siano effettivamente raggiunti o per rilevare conflitti (Lamsweerde, 2001).

Spesso è utile raccogliere e organizzare i requisiti da diverse prospettive o punti di **vista.** Diverse parti interessate possono avere diversi insiemi di requisiti. Diversi problemi di qualità possono anche portare a diversi insiemi di requisiti, portando ad esempio a un punto di vista della sicurezza. Quest'ultimo tipo di prospettiva viene solitamente trattato durante la progettazione dell'architettura software e viene discusso nel **Capitolo 11**. Le tecniche discusse nel **Capitolo 10** denotano implicitamente anche diversi punti di vista, come un punto di vista dei dati nei modelli entità-relazione. Qui, ci concentriamo su diversi punti di vista causati da diversi stakeholder. Questi diversi punti di vista possono essere in conflitto e questi conflitti devono essere riconosciuti e affrontati durante l'ingegneria dei requisiti. Il sistema di dispacciamento computerizzato per il London Ambulance Service fornisce ancora una volta un chiaro caso di punti di vista contrastanti: la direzione vuole un sistema efficace; i membri dell'equipaggio vogliono tornare a casa entro un tempo ragionevole dopo la fine del loro turno (vedi anche **Sezioni 1.4.3** e **9.1.1**). Anche per il nostro sistema bibliotecario possono verificarsi conflitti tra le parti interessate.

Si consideri, ad esempio, il seguente problema che potrebbe emergere durante la fase di elicitazione dei requisiti per il nostro sistema bibliotecario. Il sistema deve offrire alcune funzionalità per la registrazione e la gestione delle multe. Un articolo non restituito in tempo incorre in una multa, diciamo, di $ 0,25 al giorno. John, uno dei dipendenti della biblioteca coinvolti nel processo di specificazione, assume la seguente posizione (indicata con 'Pos A' nella **Figura 9.7**): i membri dovrebbero essere avvertiti delle multe in sospeso il prima possibile. La sua argomentazione ("Arg A") è che il servizio è degradato se un membro non può prendere in prestito un articolo perché un altro membro non ha restituito quell'articolo in tempo. Mary, la responsabile della biblioteca, assume una posizione piuttosto diversa ("Pos B"): i membri *non dovrebbero* essere avvertiti delle multe in sospeso fino a quando la data di scadenza non è scaduta un mese. La sua argomentazione ("Arg B") è che le multe sono un'aggiunta molto gradita al bilancio della biblioteca, che è sottoposto a forti pressioni a causa del continuo aumento dei prezzi degli abbonamenti ai giornali.

Questa situazione è rappresentata nel grafico della **Figura 9.7** . Il grafico contiene nodi di tipo "problema", "posizione" e "argomento" e collegamenti diretti di tipo "risposta a", "preso da" e "supporti". L'acquisizione di questo tipo di informazioni in un sistema automatizzato offre la possibilità di archiviare, tracciare e manipolare i diversi tipi di informazioni raccolte durante la fase di ingegneria dei requisiti. Uno dei primi sistemi in questo senso è gIBIS, un sistema progettato per acquisire le decisioni di progettazione all'inizio del processo.

Due punti di vista in particolare sono importanti durante l'ingegneria dei requisiti: il punto di vista aziendale e il punto di vista personale. Il punto di vista aziendale viene solitamente diffuso dagli stakeholder di gestione, mentre il punto di vista personale viene solitamente diffuso dagli utenti finali. Tuttavia, gli utenti finali tendono anche ad attribuire ai requisiti aziendali, almeno in una fase iniziale. Ad esempio, quando viene chiesto a John se le multe sono un'aggiunta gradita al sussidio che la biblioteca riceve dal governo, una probabile

risposta è "sì". Questo requisito è visto come un requisito dell'azienda, non un requisito personale di John. Solo quando si confronta con le conseguenze si rende conto che, dopo tutto, non è quello che vuole. E seguirà una richiesta di modifica del sistema.

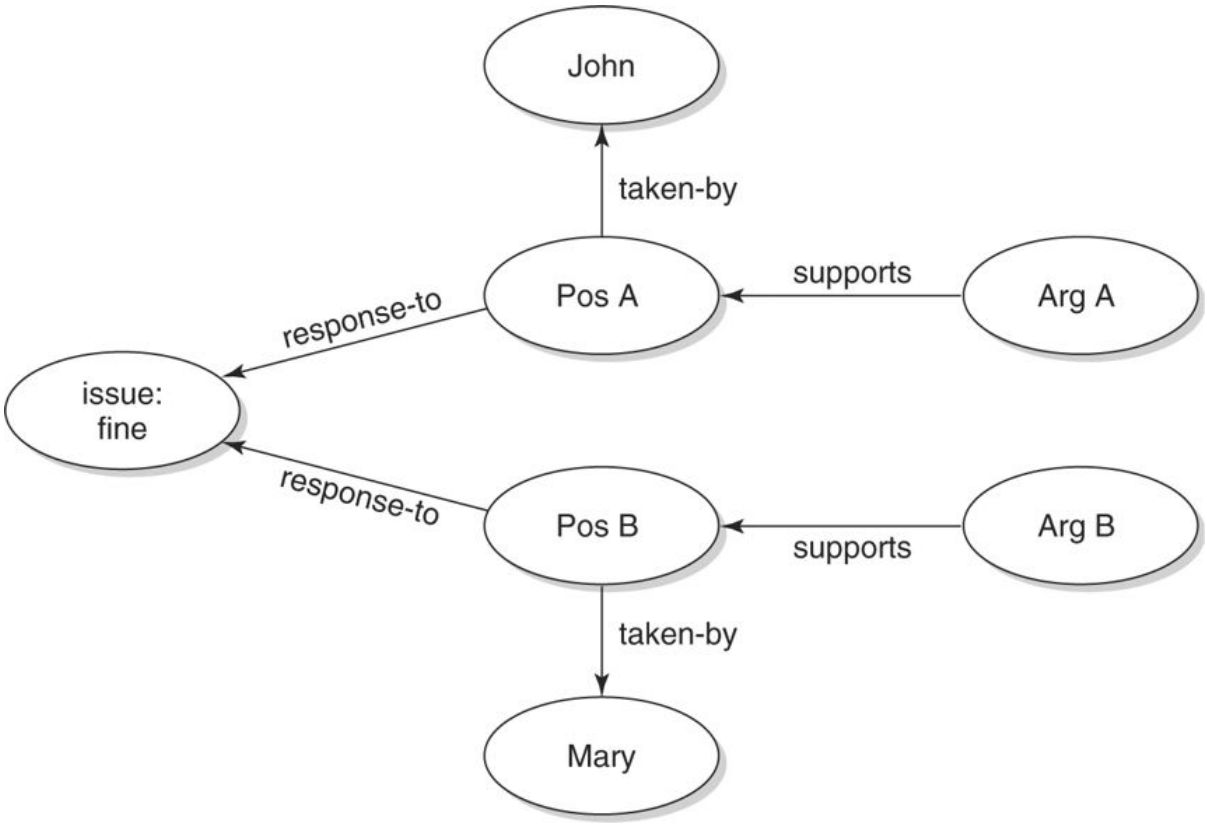


Figure 9.7 Rappresentazione di punti di vista contrastanti

9.1.4 Dare priorità ai requisiti

- Il nostro compito non è fornire tutti i pulsanti e i menu a discesa che i nostri clienti richiedono, ma inventare un modo di lavorare completamente nuovo, che li entusiasmi e li stupisca.
  Robertson (**2002**)

Nella maggior parte dei casi, non tutti i requisiti possono essere realizzati, quindi dobbiamo fare una selezione. Nella **Sezione 3.2.3** abbiamo menzionato una forma molto semplice di definizione delle priorità dei requisiti chiamata *triage.* Una variante spesso usata è conosciuta come MoSCoW (gli O sono lì solo per poter pronunciare la parola). Utilizzando MoSCoW, distinguiamo quattro tipi di requisiti:

- **Must have:** questi sono i requisiti prioritari, quelli che sicuramente devono essere realizzati per rendere il sistema accettabile per il cliente.
- **Should have, Dovrebbe avere:** questi requisiti non sono strettamente obbligatori, ma sono altamente desiderabili.

- **Could have, Potrebbe avere:** se il tempo lo consente, anche questi requisiti verranno realizzati. In pratica, di solito non lo faranno.
- **Won't have, Non ha:** questi requisiti non saranno realizzati nella versione attuale. Sono registrati però. Saranno presi in considerazione di nuovo per una versione futura del sistema.

Lo schema MoSCoW presuppone che i requisiti possano essere ordinati lungo un unico asse e che la realizzazione di più requisiti produca clienti più soddisfatti. La realtà è spesso più complessa. Nel **modello di Kano** di (Kano, **1993**), le preferenze dell'utente sono classificate in cinque categorie, come elencato nella **Tabella 9.1** . Il modo in cui i clienti valutano le categorie di requisiti Attraente, Indispensabile e Unidimensionale è illustrato nella **Figura 9.8** . Questa figura mostra che l'offerta di caratteristiche cosiddette killer attraenti è ciò che ecciterà davvero i tuoi clienti. La citazione sopra di Robertson (**2002**) punta nella stessa direzione: stupisci il tuo cliente regalandogli qualcosa che non si sarebbe mai nemmeno sognato.

Table 9-1 Categorie dei requisiti di Kano

Attraente	Il cliente è più soddisfatto se il requisito (ad esempio emettere un avviso automatico all'arrivo di nuovi libri di un amato autore) è soddisfatto, ma non meno soddisfatto se non lo è.
Deve esserci	Il cliente è insoddisfatto se il requisito (ad esempio la possibilità di effettuare ricerche nel catalogo della biblioteca) non è soddisfatto, ma soddisfatto se non supera la neutralità.
Unidimensionale	La soddisfazione è proporzionale al numero di requisiti (ad esempio, modalità alternative di ricerca nel catalogo della biblioteca) soddisfatti.
Indifferente	Al cliente non interessa realmente il requisito (ad esempio, se diverse categorie di elementi della biblioteca vengono visualizzate con un colore diverso sullo schermo).
Inversione	Il giudizio del cliente sul requisito è l'opposto di quello che si aspettava l'analista. Ad esempio, l'analista potrebbe aver pensato che il cliente della biblioteca avrebbe voluto che il sistema ricordasse i suoi modelli di ricerca in modo da poterla servire meglio la prossima volta, mentre il cliente desidera ricominciare da capo ogni volta.
Discutibile	Le preferenze del cliente non sono chiare. Sembra che le piaccia e non le piaccia una certa caratteristica.

Nello sviluppo di software orientato al mercato, il prodotto ha spesso una serie di versioni. L'elenco dei requisiti per tali prodotti è generalmente derivato da informazioni di vendita, registri utente di versioni precedenti del sistema e altre fonti di informazioni indirette. Si deve quindi decidere quali requisiti includere nella versione attuale e quali rimandare a una futura. Per stabilire le priorità vengono utilizzate analisi di casi aziendali, stime del ritorno sull'investimento e argomentazioni simili basate sull'economia. Questa impostazione di priorità deve essere ripetuta per ogni versione, poiché le preferenze dell'utente possono cambiare, il mercato reagisce e così via.

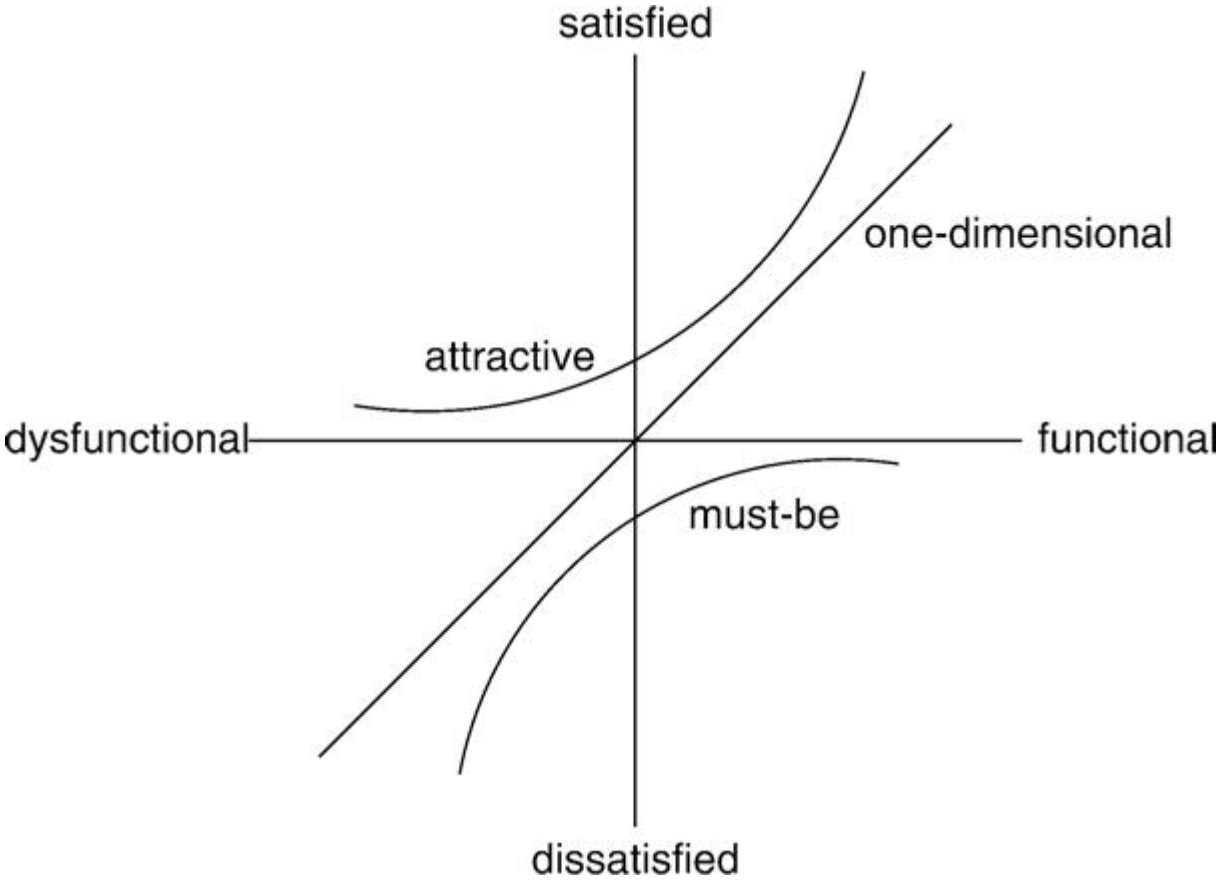


Figure 9.8 Il diagramma di Kano

Infine, la definizione delle priorità dei requisiti è correlata alla nozione di *scoping* nelle linee di prodotti software. Se vogliamo sviluppare una serie di sistemi bibliotecari simili, dobbiamo delimitare il dominio che intendiamo gestire. Un dominio più piccolo, diciamo solo biblioteche scientifiche, è più facile da realizzare, ma ha un mercato più piccolo. Un insieme di prodotti che copre un dominio più ampio è più difficile da realizzare, ma promette vendite e profitti maggiori.

9.1.5 Selezione dei COTS

Finora abbiamo affrontato una situazione in cui il cliente formula i requisiti, dopodiché viene sviluppato un sistema che soddisfa questi requisiti. Con il software COTS (commercial off-the-

shelf), il cliente deve scegliere tra ciò che è disponibile. In pratica, la situazione non è sempre così netta e un sistema COTS può essere esteso o adattato alle esigenze del cliente. Per la nostra discussione, assumiamo che sia un puro processo di selezione.

La selezione del COTS è un processo iterativo che comprende i seguenti passaggi:

1. Definisci i requisiti. Come nei normali processi di elicitazione dei requisiti, si ricava un elenco di requisiti per il prodotto. Qualsiasi delle tecniche di elicitazione discusse nella **Sezione 9.1.2** può essere utilizzata in questo processo.
2. Seleziona i componenti. Viene determinato un insieme di componenti in grado di gestire i requisiti posti. Questo processo di selezione può comportare ricerche di mercato, navigazione in Internet e una varietà di altre tecniche.
3. Classifica i componenti. I componenti sono classificati nell'ordine in cui soddisfano i requisiti.
4. Seleziona il componente più appropriato o ripeti la procedura.

Spesso, l'insieme di componenti e requisiti è troppo ampio per rendere fattibile un'analisi completa e una classificazione in un solo passaggio. Segue quindi un processo iterativo, in cui i requisiti più importanti vengono utilizzati per effettuare una prima selezione dall'insieme dei componenti disponibili. In una fase successiva, un elenco più ampio di requisiti viene valutato rispetto a un insieme più piccolo di componenti. E così via.

Esistono diversi modi per classificare i componenti. Un metodo semplice è il metodo del punteggio ponderato (WSM). A ciascun requisito viene assegnato un peso e alle alternative viene assegnato un punteggio per ciascun requisito, ad esempio su una scala da 1 a 5. Nella **tabella 9.2** , tre componenti etichettati A, B e C sono valutati in base a tre criteri: prestazioni, reputazione del fornitore, e funzionalità. Nell'esempio, i componenti B e C ottengono il punteggio più alto e questi potrebbero essere successivamente esaminati ulteriormente.

Uno dei principali svantaggi di WSM è che ogni criterio può essere compensato da qualsiasi altro criterio. Nell'esempio della **Tabella 9.2** , il componente C passa al round successivo anche se ha un punteggio molto basso in termini di funzionalità. Schemi di classificazione più complessi, come il processo di gerarchia analitica (AHP) superano questo inconveniente (Saaty, **1990**).

Table 9-2 Metodo di punteggio ponderato

Criterio	Il peso	A	B	C
Prestazione	2	1	3	5
Fornitore	1	2	2	5

Criterio	Il peso	A	B	C
Funzionalità	3	4	5	1
Totale		16	23	18

9.1.6 Crowdsourcing

Molti anni fa, con mio figlio Jasper andavo in un negozio di giocattoli per sfogliare gli scaffali con le scatole LEGO e sceglierne una che ci piaceva, per costruire un castello, un'autopompa o un aeroplano. Al giorno d'oggi, i bambini (e probabilmente anche i loro padri) visitano il sito web LEGO. Questo sito fornisce uno strumento CAD, in modo che tu possa progettare il tuo modello personalizzato del tuo edificio o auto preferito. Una volta terminato, si preme un pulsante per generare una distinta base, dopodiché i componenti necessari vengono raccolti, inseriti in una scatola e spediti. Puoi lasciare il modello creato in una galleria. Invece di creare un modello da zero, puoi sfogliare la galleria e scegliere un modello da lì. I modelli più spesso selezionati vengono preconfezionati e spediti ai negozi di giocattoli.

In questo modo, LEGO non ha bisogno di "ingegneri dei requisiti", persone che sviluppano nuovi modelli e sperano che i loro clienti ne siano entusiasti. Piuttosto, il processo di ingegneria dei requisiti è affidato a una vasta comunità di volontari: i bambini (e i loro padri). Si chiama **crowdsourcing.**

Il crowdsourcing come modello di business non è raro. Wikipedia funziona in questo modo. Le aziende possono creare un portale in cui gli utenti possono suggerire e votare miglioramenti del prodotto per guidare l'innovazione. Nell'area del software, le aziende a volte si affidano alle comunità open source per le innovazioni. La comunità open source funge quindi da filtro: i requisiti che contano filtrano e diventano parte del prodotto open source. Successivamente, la società commerciale incorpora il software open source come parte del proprio prodotto.

Piuttosto che aspettare passivamente che accada qualcosa di interessante, potremmo anche provare a indurre una folla a collaborare. Questo di solito viene fatto fornendo un kernel o una piattaforma da cui gli utenti possono sviluppare idee e prodotti. Un utente non deve "inventare" l'aspetto di un'enciclopedia online, ma può contribuire immediatamente al framework Wikipedia esistente. Allo stesso modo, molti progetti open source partono da un kernel sviluppato da una comunità molto piccola.

9.2 DOCUMENTAZIONE E GESTIONE DEI REQUISITI

Il prodotto finale della fase di ingegneria dei requisiti in un progetto di sviluppo basato su documenti è una specifica dei requisiti. La specifica dei requisiti è una ricostruzione a posteriori dei risultati della fase di analisi.

La specifica dei requisiti cambia durante e dopo la fase dei requisiti. Queste modifiche devono essere gestite.

9.2.1 Specifiche dei requisiti

Lo scopo della specifica dei requisiti è di comunicare ad altri i risultati della fase di analisi. Serve come punto di ancoraggio rispetto al quale possono essere giustificati i passaggi successivi.

La specifica dei requisiti è anche il punto di partenza per la fase successiva: la progettazione. Di conseguenza, è preferibile una descrizione molto precisa, anche matematica. D'altra parte, la specifica deve anche essere comprensibile per l'utente. Questo spesso significa un documento leggibile, usando il linguaggio naturale e le immagini. In pratica bisogna cercare un compromesso. In alternativa, la specifica dei requisiti può essere presentata in forme diverse, ma coerenti, ai diversi destinatari coinvolti.

Oltre alla leggibilità e alla comprensibilità, si possono dichiarare vari altri requisiti per questo documento (IEEE830, **1998**):

- Dovrebbe essere ***corretto***. Non esiste una procedura per garantire la correttezza. La specifica dei requisiti dovrebbe essere convalidata rispetto ad altri documenti (superiori) e alle effettive esigenze degli utenti per valutarne la correttezza.
- Dovrebbe essere ***univoco***, sia per chi lo crea che per chi lo usa. Dobbiamo essere in grado di interpretare in modo univoco i requisiti. A causa della sua stessa natura, questo è difficile da realizzare in un linguaggio naturale.
- Dovrebbe essere ***completo***. Dovrebbe documentare tutte le questioni significative relative a funzionalità, prestazioni, vincoli e così via. Dovrebbero essere specificate le risposte sia all'input corretto che a quello errato; frasi come 'da determinare' sono particolarmente insidiose. Sfortunatamente, non è sempre possibile completare le specifiche in una fase iniziale. Se alcuni requisiti possono essere specificati solo in una fase successiva, la specifica dei requisiti dovrebbe almeno documentare il momento finale in cui ciò sarebbe dovuto accadere.
- Dovrebbe essere (internamente) ***coerente***, cioè parti diverse di esso non dovrebbero essere in conflitto tra loro. I requisiti in conflitto possono essere sia logici che temporali. Anche l'uso di termini diversi per lo stesso oggetto può portare a conflitti.
- Dovrebbe **classificare** i requisiti per *importanza* o *stabilità*. In genere, alcuni requisiti sono più importanti di altri. In alcuni casi, sarà sufficiente un semplice schema di classificazione (come 'essenziale', 'utile' e 'opzionale'); in altri casi può essere necessario uno schema di classificazione più sofisticato (cfr. anche la **Sezione 9.1.4**). Possiamo indicare la stabilità

dei requisiti indicando la probabilità, o il numero previsto, di modifiche. Attraverso l'esplicita incorporazione di questo tipo di informazioni nel documento dei requisiti, gli utenti sono stimolati a tenere maggiormente in considerazione ogni requisito. Offre inoltre agli sviluppatori l'opportunità di dirigere meglio la loro attenzione.

- Dovrebbe essere ***verificabile***. Ciò significa che ci deve essere un processo finito per determinare se i requisiti sono stati soddisfatti o meno. Frasi come "il sistema dovrebbe essere di facile utilizzo" non sono verificabili. Allo stesso modo, dovrebbe essere evitato l'uso di quantità che non possono essere misurate, come in "il tempo di risposta del sistema dovrebbe essere generalmente inferiore a due secondi". È verificabile un requisito del tipo 'per le richieste di tipo X, il tempo di risposta del sistema è inferiore a due secondi nell'80% dei casi, con un carico macchina massimo di Y'.
- Dovrebbe essere ***modificabile***. Il software modella una parte della realtà. Quindi cambia. La specifica dei requisiti corrispondenti deve evolversi con la realtà modellata. Pertanto, il documento deve essere organizzato in modo tale che le modifiche possano essere adattate prontamente (un formato tabellare o di database, ad esempio). La ridondanza deve essere evitata il più possibile, altrimenti c'è il pericolo che i cambiamenti portino a incoerenze.
- Dovrebbe essere ***tracciabile***. L'origine e la motivazione di ogni esigenza deve essere rintracciabile. Uno schema di numerazione chiaro e coerente consente ad altri documenti di fare riferimento in modo univoco a parti della specifica dei requisiti.

Come linea guida per il contenuto di una specifica dei requisiti, seguiremo lo standard IEEE 830. Questo standard non fornisce una forma rigida per la specifica dei requisiti. A nostro avviso, l'ordine preciso e il contenuto degli elementi di questo documento sono tutt'altro che essenziali. Il punto importante è scegliere una struttura che aderisca ai vincoli di cui sopra. (IEEE830, **1998**) utilizza una struttura globale come quella illustrata nella **Tabella 9.3**.

Table 9-3 Struttura globale della specifica dei requisiti

1.	introduzione
	1.1 Obiettivo
	1.2 Scopo
	1.3 Definizioni, acronimi e abbreviazioni
	1.4 Riferimenti
	1.5 Panoramica
2.	Descrizione generale
	2.1 Prospettiva del prodotto
	2.2 Funzioni del prodotto
	2.3 Caratteristiche dell'utente
	2.4 Vincoli

2.5 Presupposti e dipendenze 3. Requisiti specifici

(*Fonte: Pratica consigliata IEEE per la specifica dei requisiti software*, Standard IEEE 830, © **1998** . Riprodotto con il permesso di IEEE.)

Per qualsiasi sistema non banale, i requisiti dettagliati costituiranno di gran lunga la parte più ampia del documento dei requisiti. È quindi utile classificare in qualche modo questi requisiti dettagliati. Questo può essere fatto lungo diverse dimensioni, come ad esempio:

- **Modalità:** i sistemi possono comportarsi in modo diverso a seconda della modalità operativa, ad esempio di addestramento o operativa. Ad esempio, i requisiti delle prestazioni o dell'interfaccia possono differire tra le modalità.
- **Classe utente:** funzionalità diverse possono essere offerte a classi diverse di utenti, come i membri della biblioteca e il personale della biblioteca.
- **Oggetti:** i requisiti possono essere classificati in base agli oggetti (entità del mondo reale) interessati. Questo schema di classificazione è naturale se usato insieme a una tecnica di analisi orientata agli oggetti (vedi **Sezione 12.3**).
- **Risposta:** alcuni sistemi sono meglio descritti mettendo insieme funzioni a supporto della generazione di una risposta, ad esempio funzioni associate a query di catalogo o informazioni sullo stato dei membri della biblioteca.
- **Gerarchia funzionale:** quando non si adatta un'altra classificazione, è possibile utilizzare una gerarchia funzionale, ad esempio organizzata da input comuni.

A titolo di esempio, la **Tabella 9.4** fornisce un perfezionamento della sezione sui requisiti specifici lungo la dimensione delle classi di utenti. La **Figura 9.9** contiene (parte di) una possibile specifica dei requisiti per l'esempio di libreria, seguendo le linee guida IEEE.

Table 9-4 Schema della sezione Requisiti Specifici

3. Requisiti specifici 3.1 Requisiti dell'interfaccia esterna 3.1.1 Interfacce utente 3.1.2 Interfacce hardware 3.1.3 Interfacce software 3.1.4 Interfacce di comunicazione 3.2 Richieste funzionali

3.2.1 Classe utente 1

3.2.1.1 Requisito funzionale 1.1

3.2.1.2 Requisito funzionale 1.2

…

3.2.2 Classe utente 2

…

3.3 Requisiti di prestazione

3.4 Vincoli di progettazione

3.5 Attributi del sistema software

3.6 Altri requisiti

(*Fonte: IEEE Recommended Practice for Software Requirements Specifications*, IEEE Standard 830, © **1998.** Riprodotto con il permesso di IEEE.)

Il framework IEEE per la specifica dei requisiti è particolarmente appropriato nei modelli basati su documenti per il processo di sviluppo del software: il modello a cascata e le sue varianti. Quando una tecnica di prototipazione viene utilizzata per determinare l'interfaccia utente, il framework IEEE può essere utilizzato per descrivere il risultato di quel processo di prototipazione. Il framework presuppone un modello in cui il risultato del processo di ingegneria dei requisiti è inequivocabile e completo. Sebbene si affermi che i requisiti dovrebbero essere classificati in base all'importanza e che i requisiti che possono essere posticipati fino a versioni future possono essere inclusi come sottoinsiemi, ciò non implica che una visione stratificata del sistema possa essere facilmente derivata da un documento dei requisiti redatto in questo modo.

Indipendentemente dal formato scelto per rappresentare i requisiti, il successo di un prodotto dipende fortemente dal grado in cui il sistema desiderato viene adeguatamente descritto durante la fase di ingegneria dei requisiti. Piccoli errori nelle specifiche dei requisiti possono richiedere grandi cambiamenti nel software finale. Nel **Capitolo 1** , lo abbiamo descritto dicendo che il software non è continuo.

1. *Introduction*.

1.1 *Purpose*. This document states the requirements of an automated library system for a medium-sized library of a research institute. The requirements stated serve as a basis for the acceptance procedure of this system. The document is also intended as a starting point for the design phase.

1.2 *Scope*. The intended product automates the library functions described in DOC1. Its purpose is to provide a more effective service to the library users, in particular through the online search facilities offered. More details of the performance requirements are given in section 3.3 of this document. Once this system is installed, the incorporation of new titles will go from an average of 15 minutes down to an average of 5 minutes.

1.3 *Definitions, acronyms and abbreviations*. Library member: . . . , Library personnel: . . . , User: The term user may refer to both library members and library personnel, and is used to denote either class of user. Title catalog: . . . , PICA: . . . , etc.

1.4 *References*. DOC1: . . . , DOC2: . . . , etc.

1.5 *Overview*. Section 2 of this document gives a general overview of the system. Section 3 gives more specific requirements for functions offered. These functions are categorized according to the class of users they support: (external) members of the library and library personnel, respectively.

2. *Overall description*.

2.1 *Product perspective*. The already installed database system X will be used to store the various catalogs as well as the library member administration. There are no interfaces to other systems. The system will be realized on the Y configuration. The maximum external storage capacity for the catalogs of the system is 1500 MB. Library personnel will use a bar-code reader to enter member, book and journal identifications. The interface protocol to the bar-code reader is described in DOC4.

2.2 *Product functions*. The system provides two types of function:

- Functions by which users may search the catalogs of books and journal articles. A list of these functions is given in DOC1. A more detailed description is given in Section 3.2.1.
- Functions by which library personnel may update the administration of borrowed titles and the system's catalogs; see section 3.2.2.

The user of the system selects one of the functions offered through the main menu (section 3.2.1.1 and 3.2.2.1).

2.3 *User characteristics*. The library members are incidental users of the system and have little knowledge of automated systems of this kind. The system therefore has to be self-instructing. Specific requirements are formulated in sections 3.1.1 and 3.3. The library personnel will be trained in the use of the system; see section 3.1.1.

Figure 9.9 Specifica dei requisiti parziali per l'esempio di libreria

2.4 *Constraints*. Library members may only search the catalogs of books and journal articles. They are not allowed to update a catalog or access the user administration. The latter functionality is to be offered only through a dedicated, password-protected interface.

2.5 *Assumptions and dependencies*. . . .

3. *Specific requirements*.

3.1 *External interface requirements*.

3.1.1 *User interfaces*. The screen formats for the different features are specified in Appendix A. Appendix B lists the mapping of commands to function keys. The user can get online help at any point by giving the appropriate command. Appendix C contains a list of typical usage scenarios. These usage scenarios will be used as acceptance criteria: 80% of the users must be able to go through them within 10 minutes. An instruction session for library personnel should take at most two hours.

3.1.2 *Hardware interfaces*. The user interface is screen-oriented. The system uses up to ten function keys.

3.1.3 *Software interfaces*. The interface with database system X is described in DOC2.

3.1.4 *Communications interfaces*. Not applicable.

3.2 *Functional requirements*.

3.2.1 *Library member functions*.

3.2.1.1 *Select member feature*. The user selects one of the options from the main menu. Subsequent actions are described in sections 3.2.1.2 and 3.2.1.3. At any point, the user has an option to return to the main menu (see Appendix B).

3.2.1.2 *Search book catalog*. Given (part of) a book title or author name, the user may search the book catalog for titles that match the input given. The user is offered a screen with two fill-in-the-blank areas (one for the title and one for the author), one of which is to be filled in.

Input. The input may contain both upper- and lower-case letters. Special symbols allowed are listed in DOC1. Any other glyphs entered are discarded and are not shown on the screen. The input is considered complete when the processing command is issued.

Processing. All lower-case letters are turned into upper-case letters. The string thus obtained is used when querying the database. A database entry matches the title string given if the transformed input is a substring of the title field of the entry. The same holds for the author field if (part of) an author name is input.

Output. A list of titles that match the input is displayed. Up to four titles are shown on the screen. The user may traverse the list of titles found using the screen-scrolling commands provided. A warning is issued if no title matches the input given.

3.2.1.3 *Search article catalog*. . . .

3.2.2 *Library personnel functions.*

3.2.2.1 *Select personnel feature.* Through a dedicated, password-protected interface, library personnel are offered an extended main menu, listing the options available to all users and the options available only to library personnel. The latter are described in sections 3.2.2.2 and 3.2.2.3.

3.2.2.2 *Borrow title.* . . .

3.2.2.3 *Modify catalog.* . . .

3.3 *Performance requirements.* The system will initially support 32 concurrent access points. Its maximum capacity is 128 concurrent access points.

The present database holds 25 000 book titles and 500 journal subscriptions. The storage capacity needed for data is 300 MB. On average 1 000 books and 2 000 journal issues enter the library per year. The average journal issue has six articles. This requires a storage capacity of 15 MB per year.

The system must be able to serve 20 users simultaneously. With this maximum load and a database size of 450 MB, user queries as listed in sections 3.2.1 and 3.2.2 must be answered within five seconds in 80% of the cases.

3.4 *Design constraints.*

3.4.1 *Standards compliance.* Title descriptions must be stored in PICA format. This format is described in DOC3.

3.4.2 *Hardware limitations.* See section 2.1.

3.5 *Software system attributes.*

3.5.1 *Availability.* During normal office hours (9 am–5 pm) the system must be available 95% of the time. A backup of the system is made every day at 5 pm.

3.5.2 *Security.* The functions described in Section 3.2.2 are restricted to library employees and are password-protected. . . .

3.5.3 *Maintainability.* . . .

3.6 *Other requirements.* . . .

L'importanza di una solida specifica dei requisiti non può essere sottolineata abbastanza spesso. In alcuni casi, è stato necessario riscrivere fino al 95% del codice di sistemi di grandi dimensioni per soddisfare i requisiti dell'utente finale.

Un caso di studio: il ventilatore meccanico polmonare

https://github.com/foselab/abz2024_casestudy_MLV/blob/main/Mechanical_Lung_Ventilator%201_5.pdf

Il caso di studio presentato in classe riguarda un ventilatore meccanico polmonare, che abbiamo progettato e realizzato durante la pandemia. Vediamo qui la struttura del documento dei requisiti. Il documento è così strutturato

9.2.2 Gestione dei requisiti

Un problema fondamentale con il framework IEEE discusso nella **Sezione 9.2.1** è che descrive solo il prodotto finale. Prima che questa fase finale sia raggiunta, l'insieme di requisiti "attuale" è in uno stato di flusso costante. E anche al termine della fase dei requisiti, i requisiti cambieranno e verranno presentati nuovi requisiti. Quest'ultimo fenomeno è noto come **requisiti creep** ed è la causa di molti progetti in fuga.

Le modifiche ai requisiti non possono essere aggirate. In molti casi, non è nemmeno saggio puntare a un blocco anticipato dei requisiti. In generale, la situazione preferita è quella illustrata nella **Figura 9.10** : nel corso del tempo, l'insieme dei requisiti diventa sempre più stabile.

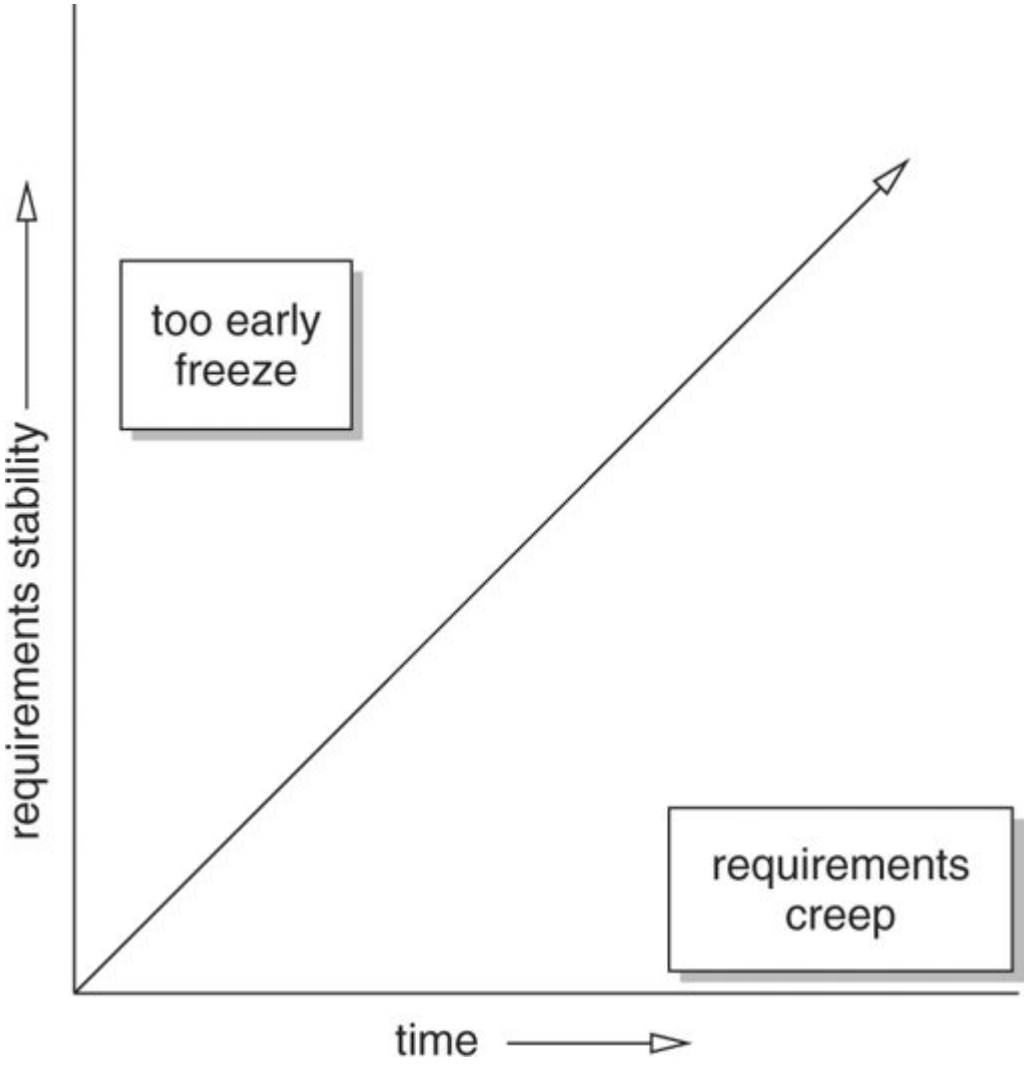


Figura 9.10 Stabilità dei requisiti nel tempo

Ovviamente, questo insieme in evoluzione di requisiti deve essere gestito. La gestione dei requisiti prevede tre attività:

- identificazione dei requisiti,
- gestione delle modifiche dei requisiti e
- tracciabilità dei requisiti.

Ogni requisito deve avere un mezzo di identificazione univoco. Il metodo più semplice è numerarli. Se esiste un'organizzazione gerarchica, come in una gerarchia di obiettivi, può riflettersi nello schema di numerazione. Poiché i requisiti vengono spesso modificati e aggiornati, è opportuno includere anche le informazioni sul controllo delle versioni. Infine, possiamo aggiungere alcuni attributi a ciascun requisito, come il suo stato, la priorità, il principale stakeholder coinvolto e così via. Gli strumenti di ingegneria dei requisiti di solito hanno un modo per archiviare i requisiti in un repository strutturato.

Le modifiche ai requisiti devono essere gestite correttamente. Visualizzando ogni requisito come un elemento di configurazione, è possibile applicare le regole e le procedure di gestione della configurazione (**Capitolo 4).**

Possiamo collegare i requisiti agli elementi della soluzione come elementi di progettazione o anche componenti software che realizzano tali requisiti. In questo modo stabiliamo la **tracciabilità** dai requisiti al codice e viceversa. Questo ci permette di tracciare dove vengono realizzati i requisiti (tracciabilità in avanti) e perché vengono scelte determinate soluzioni (tracciabilità a ritroso). Le informazioni sulla tracciabilità sono importanti in tutte le fasi di sviluppo. Può essere utilizzato per rispondere a una varietà di domande, come ad esempio:

- Dove viene implementato questo requisito?
- Abbiamo bisogno di questo requisito?
- Tutti i requisiti sono collegati agli elementi della soluzione?
- Quale requisito copre questo test case?
- Qual è l'impatto di questo requisito?
- Abbiamo bisogno di questo elemento di design (pezzo di codice)?

Questo modo di rendere esplicito il rapporto tra esigenze e soluzioni è strettamente correlato **all'analisi dello spazio progettuale.** Nell'analisi dello spazio progettuale, l'obiettivo è modellare in modo esplicito tutte le possibili combinazioni di requisiti e soluzioni. L'analisi dello spazio progettuale nasce nel campo dell'interazione uomo-computer. Una nota notazione per l'analisi dello spazio di progettazione è nota come "domande, opzioni e criteri" (QOC). Le domande corrispondono ai requisiti; le opzioni sono le possibili risposte a tali requisiti; ei criteri si riferiscono alle ragioni per la scelta di una particolare opzione. Ad esempio, potremmo visualizzare il risultato di una richiesta di notizie da parte di un cliente della biblioteca (la domanda) ordinato per data di pubblicazione o per nome dell'autore (le opzioni). Il criterio per l'utilizzo di un ordinamento specifico potrebbe essere la fonte della notizia; ad esempio, gli articoli di giornale possono essere ordinati per data e gli articoli di giornale per nome dell'autore.

Da un lato, l'analisi dello spazio progettuale si traduce in una struttura ricca in cui viene costruito un ampio registro delle ragioni per una soluzione specifica. Perché questo sistema è costruito così com'è? Quali alternative abbiamo considerato ma rifiutato? Quali requisiti sono sopravvissuti ai compromessi che abbiamo dovuto fare? D'altra parte, acquisire tutte queste informazioni è costoso e i benefici immediati sono difficili, se non impossibili, da dimostrare. Questo è uno dei motivi principali per cui la logica progettuale, nel complesso, non è riuscita a trasferirsi nella pratica.

9.3 TECNICHE PER LA SPECIFICA DEI REQUISITI

Il documento prodotto durante l'ingegneria dei requisiti, la specifica dei requisiti, serve due gruppi di persone. Per l'utente, la specifica dei requisiti è una descrizione chiara e precisa delle

funzionalità che il sistema ha da offrire. Per il designer, è il punto di partenza per il design. Non è facile servire entrambi i gruppi con un unico documento.

9.3.1 Scelta di una notazione

L'utente, in generale, è meglio servito da un documento che parli la sua lingua, la lingua utilizzata all'interno del dominio dell'applicazione. Nell'esempio della libreria, ciò comporterebbe l'utilizzo di termini come "descrizione del titolo" e "catalogo".

Il designer, d'altra parte, è meglio servito da un linguaggio in cui vengono utilizzati concetti del suo mondo. Per quanto riguarda l'esempio della biblioteca, potrebbe preferire concetti come "record" (un esempio del quale potrebbe essere definito "descrizione del titolo") o "file". In un certo senso, questo si riduce a una differenza di lingua. Tuttavia, questa differenza è di fondamentale importanza rispetto all'uso successivo della descrizione del sistema.

Se il sistema è descritto nella lingua dell'utente, la specifica dei requisiti è per lo più formulata in **un linguaggio naturale**. Se proviamo a formalizzare in qualche modo questa descrizione, potremmo finire con una tecnica in cui devono essere compilati determinati moduli o devono essere applicate determinate tecniche di disegno.

Se, invece, gioca un ruolo centrale il linguaggio esperto dell'ingegnere del software, spesso utilizziamo un **linguaggio formale**. Una specifica dei requisiti formulata in un tale linguaggio formale può essere verificata utilizzando tecniche formali, ad esempio per quanto riguarda la coerenza e la completezza.

In pratica si manifesta una forte prevalenza del linguaggio esperto dell'utente. Possiamo quindi utilizzare concetti esistenti dall'ambiente in cui verrà utilizzato il sistema. È vero che questi concetti non sono definiti in modo netto, ma in generale non ci sono idee sbagliate tra gli esperti nel campo di applicazione per quanto riguarda il significato di tali concetti. Una descrizione in termini di tali concetti può quindi essere ancora molto *precisa.* Poiché il primo obiettivo della specifica dei requisiti è ottenere una descrizione *completa* del problema da risolvere, il linguaggio esperto dell'utente può essere il linguaggio migliore per la specifica dei requisiti.

Tuttavia, ci sono alcuni inconvenienti legati all'uso del linguaggio naturale. Meyer (**1985**) fornisce un esempio che illustra molto bene cosa potrebbe andare storto quando il linguaggio naturale viene utilizzato in una specifica dei requisiti. Meyer elenca **sette** peccati che possono affliggere l'analista quando usa il linguaggio naturale:

1. **Rumore:** Si riferisce alla presenza di elementi di testo che non contengono informazioni rilevanti per il problema. Le varianti del rumore sono la ridondanza e il rimpianto. *La ridondanza* si verifica quando le cose si ripetono. Poiché il linguaggio naturale è molto flessibile, le questioni correlate possono essere facilmente formulate in modi completamente diversi. Quando ciò accade, la coesione tra le cose si confonde. *Il rimpianto* si verifica quando le istruzioni sono invertite o ombreggiate. Nell'esempio della

biblioteca, ad esempio, avremmo potuto usare più volte la frase "un elenco di tutti i libri scritti dall'autore D" e solo allora renderci conto che questo elenco potrebbe essere vuoto, rendendo necessaria una reazione speciale da parte del sistema.

2. **Silenzio:** Il silenzio si verifica quando non vengono menzionati aspetti importanti per una corretta soluzione del problema. Un esempio di ciò potrebbe essere che la necessità di due varianti del nome di un autore non è stata dichiarata esplicitamente.
3. **Sovraspecificazione:** si verifica quando gli elementi di una specifica dei requisiti corrispondono ad aspetti di una possibile soluzione, piuttosto che ad aspetti del problema. Ad esempio, avremmo potuto specificare che i libri devono essere tenuti ordinati in base al nome del primo autore. L'eccesso di specifiche limita lo spazio della soluzione per il progettista.
4. **Contraddizioni:** se la descrizione di un aspetto viene data più di una volta, con parole diverse, possono verificarsi delle contraddizioni. Questo rischio è particolarmente minaccioso quando si cerca di essere troppo letterari. Una specifica dei requisiti non è pensata per essere un romanzo.
5. **Ambiguità:** il linguaggio naturale consente più di un significato per ogni frase. L'ambiguità può facilmente verificarsi quando vengono utilizzati termini che appartengono al gergo di una o entrambe le parti. Un "libro" può denotare sia un oggetto fisico che un'entità più astratta di cui possono esistere diverse istanziazioni (copie).
6. **Riferimenti diretti:** Un riferimento diretto si riferisce ad aspetti del problema che sono definiti solo più avanti nel testo. Ciò si verifica soprattutto nei documenti di grandi dimensioni privi di una struttura chiara. Il linguaggio naturale di per sé non impone una struttura chiara.
7. **Pio desiderio (wishful thinking):** alcune descrizioni di aspetti del sistema sono tali che sarà difficile trovare una soluzione realistica.

Una possibile alternativa data da Meyer consiste nel descrivere e analizzare prima il problema utilizzando una notazione formale e poi tradurlo nuovamente nel linguaggio naturale. La descrizione in linguaggio naturale così ottenuta rappresenterà, in generale, una nozione più precisa del problema che è leggibile dall'utente. Ovviamente, entrambi questi modelli devono essere tenuti aggiornati.

Numerose tecniche e annotazioni di accompagnamento si sono evolute per supportare il processo di ingegneria dei requisiti. Molto spesso, la rappresentazione generata è un insieme di reti semantiche. Ciascuna di queste rappresentazioni ha vari tipi di nodi e collegamenti tra i nodi, distinti da indizi visivi come la loro forma o etichette del linguaggio naturale. I nodi in genere rappresentano cose come processi, negozi, oggetti e attributi. I nodi sono uniti da frecce che rappresentano relazioni come flusso di dati, flusso di controllo, astrazione, parte-tutto o parte di.

Esempi tipici di tali tecniche e delle loro rappresentazioni sono discussi nel **Capitolo 10** . La modellazione entità-relazione (**Sezione 10.1.1**) è una tecnica ampiamente nota per modellare l'aspetto dei dati di un sistema informativo. Le macchine a stati finiti (**Sezione**

10.1.2) possono essere viste come una tecnica per modellare l'aspetto funzionale. Tuttavia, hanno un'applicabilità molto più ampia e costituiscono un meccanismo di base di base per molte tecniche di modellazione. In particolare, l'Unified Modeling Language (UML) deve a loro omaggio. I diagrammi UML (**Sezione 10.3**) sono ampiamente utilizzati per modellare il risultato dell'ingegneria dei requisiti e della progettazione.

9.3.2 Specifica dei requisiti non funzionali

Il framework IEEE illustrato nella **Tabella 9.4** elenca quattro tipi di requisiti non funzionali:

1. requisiti di interfaccia esterna,
2. requisiti di prestazioni,
3. vincoli di progettazione e
4. attributi del sistema software.

Questi requisiti non funzionali possono essere visti come vincoli imposti al processo di sviluppo o ai prodotti da consegnare.

I requisiti dell'interfaccia esterna e i vincoli di progettazione sono generalmente espressi in termini di obblighi (non negoziabili) da soddisfare. Sono dettati all'inizio del progetto e spesso riguardano questioni che superano un progetto di sviluppo individuale. Esempi di tali requisiti includono:

- hardware, software e interfacce di comunicazione da rispettare;
- interfacce utente che devono rispettare gli standard aziendali;
- formati di report da rispettare;
- vincoli di processo come la conformità alla ISO 9000 o un metodo di sviluppo prescritto;
- limitazioni hardware causate dall'infrastruttura disponibile.

I restanti requisiti non funzionali sono anche noti come **requisiti di qualità**. I requisiti di qualità sono notoriamente difficili da specificare e verificare. Questo argomento è trattato ampiamente nel **Capitolo 6** . A questo punto, desideriamo semplicemente ribadire due questioni essenziali: i requisiti di qualità devono essere espressi in termini oggettivi e misurabili e la perfezione ha un costo infinito. Come tutti gli altri requisiti, i requisiti di qualità dovrebbero essere verificabili. Requisiti come "il sistema dovrebbe essere flessibile", "il sistema dovrebbe essere di facile utilizzo" o "i tempi di risposta dovrebbero essere rapidi", non possono mai essere verificati e pertanto non dovrebbero comparire nella specifica dei requisiti. Possono essere utilizzate altre fraseologie del tipo 'per attività di tipo A, il sistema dovrebbe avere un tempo massimo di risposta di un secondo nell'80% dei casi, mentre è consentito un tempo massimo di risposta di tre secondi nel restante 20% dei casi.'

Al contrario, livelli estremi di requisiti di qualità, come zero difetti o tempi di risposta inferiori a 1 secondo nel 100% dei casi, generalmente comportano costi estremamente elevati o non sono affatto fattibili. Dato che gli utenti hanno difficoltà a esprimere le loro reali esigenze,

possono essere inclini a chiedere troppo per quanto riguarda i requisiti di qualità, "solo per andare sul sicuro". Allo stesso modo, per l'analista e gli sviluppatori è difficile valutare la fattibilità di tali requisiti. Come possiamo essere sicuri dei tempi di risposta prima che sia stata scritta anche una sola riga di codice?

Considera il seguente esempio di ciò che può e non può essere tecnicamente fattibile. Supponiamo di avere un'applicazione in cui possono verificarsi due tipi di transazioni. Tali transazioni sono caratterizzate dalla frequenza, dal tempo di CPU necessario e dal numero di trasporti fisici di I/O. Viene fornito anche il tempo medio di accesso agli I/O. Utilizzando una distribuzione statistica che descrive la dinamica di questi sistemi, si può quindi rispondere a domande del tipo: "Quanta capacità dovrebbe avere la CPU per ottenere un tempo di risposta di al massimo 2 secondi nell'X% dei casi?" Una determinata configurazione può soddisfare i vincoli per il caso X = 80. Un requisito un po' più rigoroso (X = 90) può richiedere il raddoppio della capacità della CPU. Un requisito ancora più severo (X = 95) potrebbe non essere raggiungibile dalla gamma di macchine disponibili.

A prima vista, le differenze tra questi requisiti sembrano marginali. Tuttavia, risultano avere un effetto tremendo. Un'analisi precoce e attenta della fattibilità tecnica può fornire risposte sorprendenti a una serie di importanti domande. Di solito, questo tipo di analisi viene eseguito al momento dell'architettura del software. Ci sono molti esempi di progetti in cui sono stati spesi molti soldi per sforzi di sviluppo del software che si sono rivelati non praticamente fattibili (Baber, **1982**).

9.4 VERIFICA E CONVALIDA/VALIDAZIONE

Nel **Capitolo 1** abbiamo affermato che uno studio attento della correttezza delle decisioni prese in ogni fase è un fattore critico di successo. Ciò significa che durante l'ingegneria dei requisiti dovremmo già iniziare a verificare e convalidare le decisioni stabilite nella specifica dei requisiti.

La specifica dei requisiti dovrebbe riflettere la comprensione reciproca del problema da risolvere da parte dei potenziali utenti e dell'organizzazione di sviluppo: tutto è stato descritto ed è stato descritto correttamente? Convalidare i requisiti significa quindi verificarne proprietà quali correttezza, completezza, ambiguità e coerenza interna ed esterna. Ciò implica necessariamente la partecipazione dell'utente al processo di convalida. Sono loro i proprietari del problema e sono gli unici che possono decidere se la specifica dei requisiti descriva adeguatamente il loro problema.

Se la specifica dei requisiti stessa è espressa in un linguaggio formale, la sintassi e la semantica di tale rappresentazione possono essere verificate attraverso mezzi formali. Tuttavia, la specifica dei requisiti non può mai essere completamente convalidata in modo formale, semplicemente perché il punto di partenza dell'ingegneria dei requisiti è informale. La maggior parte delle tecniche di test applicate in questa fase sono quindi anche informali. Hanno lo scopo di accertare che le parti coinvolte abbiano la stessa, corretta comprensione

del problema. Uno dei principali ostacoli in questa fase è garantire che l'utente comprenda il contenuto della specifica dei requisiti. Le tecniche applicate in questa fase spesso si risolvono in una traduzione dei requisiti in una forma appetibile per l'ispezione dell'utente: parafrasi in linguaggio naturale, discussione di possibili scenari di utilizzo, prototipazione e animazione.

Oltre a testare la specifica dei requisiti stessa, in questa fase generiamo anche il piano di test da utilizzare durante i test di sistema o di accettazione. Un piano di test è un documento che prescrive l'ambito, l'approccio, le risorse e il programma delle attività di test. Identifica gli elementi e le caratteristiche da testare, le attività di verifica da eseguire e il personale responsabile di tali attività. A questo punto possiamo sviluppare un tale piano per la fase di test del sistema, cioè la fase in cui l'organizzazione di sviluppo verifica l'intero sistema rispetto ai suoi requisiti. Il test di accettazione è simile, ma viene eseguito sotto la supervisione dell'organizzazione dell'utente. Il test di accettazione ha lo scopo di determinare se gli utenti accettano o meno il sistema.

Una trattazione più elaborata delle varie tecniche di verifica e validazione è data nel **Capitolo 13** .

9.5 RIEPILOGO

Durante la fase di ingegneria dei requisiti, cerchiamo di ottenere una descrizione completa e chiara del problema da risolvere e dei vincoli che devono essere soddisfatti da qualsiasi soluzione a quel problema. In questa fase, non solo consideriamo le funzioni da erogare, ma prestiamo attenzione anche ai requisiti imposti dall'ambiente. La fase di ingegneria dei requisiti si traduce in una serie di modelli incentrati su diversi aspetti del sistema (come la sua funzionalità, interfaccia utente e struttura di comunicazione) e diverse prospettive (audience). Il risultato di questo processo è documentato in una specifica dei requisiti. Un buon quadro per il contenuto della specifica dei requisiti è fornito in (IEEE830, **1998**). Va tenuto presente che questo documento contiene una ricostruzione a posteriori di un processo iterativo ancora mal compreso.

Questo processo iterativo prevede quattro tipi di attività:

- elicitazione dei requisiti, che riguarda la *comprensione* del problema,
- specifica dei requisiti, che riguarda la *descrizione* del problema,
- convalida dei requisiti, che consiste nel *concordare* il problema, e
- negoziazione dei requisiti, che consiste *nell'adattare* il problema alla situazione attuale.

In molti casi, non è possibile completare completamente l'ingegneria dei requisiti prima di iniziare la progettazione e la costruzione. Nei processi di sviluppo agile, l'ingegneria dei requisiti è un processo iterativo intrecciato con la progettazione e la costruzione.

Durante l'ingegneria dei requisiti stiamo modellando parte della realtà. La parte della realtà che ci interessa è indicata come l'universo del discorso (UoD). Il processo di modellazione è chiamato modellazione concettuale.

Le persone coinvolte in un UoD hanno un modello concettuale implicito di quell'UoD. Durante la modellazione concettuale, un modello implicito viene trasformato in un modello esplicito. Il modello concettuale esplicito viene utilizzato per comunicare con altre persone, come utenti e progettisti, e per valutare la validità del sistema in fase di sviluppo durante tutte le fasi successive. Durante il processo di modellazione, l'analista si confronta con problemi di analisi e problemi di negoziazione. I problemi di analisi hanno a che fare con l'ottenimento dei requisiti corretti. I problemi di negoziazione sorgono perché diverse persone coinvolte possono avere opinioni diverse sull'UoD da modellare, interessi opposti e così via.

Gli approcci esistenti all'ingegneria dei requisiti sono in gran parte di natura tayloriana. Si adattano a una visione funzionale dello sviluppo del software in cui la fase di ingegneria dei requisiti serve a suscitare i requisiti "reali" dell'utente. È sempre più riconosciuto che l'approccio tayloriano non deve necessariamente essere l'approccio più appropriato all'ingegneria dei requisiti. Molti universi del discorso coinvolgono persone il cui modello del mondo è incompleto, irrazionale o in conflitto con la visione del mondo degli altri. In questi casi, l'analista non è un osservatore esterno passivo dell'UoD, ma partecipa attivamente alla formazione dell'UoD. L'analista viene coinvolto in problemi di negoziazione e deve scegliere il punto di vista di una parte coinvolta o aiutare a ottenere qualche compromesso.

Le seguenti tecniche di descrizione sono spesso utilizzate per la specifica dei requisiti:

- linguaggio naturale,
- immagini, e
- linguaggio formale.

Un vantaggio dell'uso del linguaggio naturale è che le specifiche sono molto leggibili e comprensibili per l'utente e altri non professionisti coinvolti. Le immagini possono avere un vantaggio nel trasmettere l'architettura funzionale del sistema. Un linguaggio formale ci consente di utilizzare strumenti nell'analisi dei requisiti. Per la sua precisione, è un buon punto di partenza per la fase di progettazione. Possiamo anche sostenere che dovrebbero essere utilizzate sia le notazioni formali che quelle informali, poiché si aumentano e si completano a vicenda. Per ciascuna delle parti coinvolte, dovrebbe essere scelta una notazione adeguata al compito da svolgere.

9.6 ULTERIORI LETTURE

Ci sono molti libri di testo interamente dedicati all'ingegneria dei requisiti. Davis (**1993**) fornisce una copertura abbastanza completa delle tecniche di specificazione dei requisiti "classici". Davis (**2005**) si concentra sull'ingegneria dei requisiti di fronte a vincoli di pianificazione serrati. Wieringa (**1996**) discute un certo numero di tecniche di specifica dei

requisiti in modo abbastanza approfondito e fa la distinzione tra modelli concettuali impliciti ed espliciti. Loucopoulos e Karakostas (**1995**) e Kotonya e Sommerville (**1997**) hanno una maggiore enfasi sull'intero processo di ingegneria dei requisiti. (Juristo *et al.*, **2002**) discutono dello stato della pratica nell'ingegneria dei requisiti. Hofman e Lehner (**2001**) si concentrano sulle pratiche di successo dei requisiti. Sommerville (**2005**) discute i recenti sviluppi nel campo.

Pohl (**1993**) e Goguen e Jirotka (**1994**) sottolineano il ruolo delle questioni sociali e cognitive nell'ingegneria dei requisiti. (Ramos *et al.*, **2005**) sostengono che l'emozione è rilevante nell'ingegneria dei requisiti. La scarsa diffusione delle conoscenze applicative tra gli specialisti coinvolti è discussa in (Curtis *et al.*, **1988**). Le difficoltà dell'ingegneria dei requisiti per lo sviluppo di software orientato al mercato sono affrontate in (Potts, **1993**). Moynihan (**2000**) discute come i manager affrontano l'incertezza dei requisiti.

Le dimensioni oggettivista-soggettivista e ordine-conflitto ei quattro paradigmi risultanti per l'ingegneria dei requisiti sono discussi in (Hirschheim e Klein, **1989**). In (Atkinson, **2000**) vengono discussi vari approcci socio-tecnici e soggettivisti all'elicitazione dei requisiti.

L'analisi del compito è discussa in (Sebillotto, **1988**). Le tecniche di ingegneria dei requisiti basate su scenari sono discusse in (Weidenhaupt *et al.*, **1998**) e (TrSE, **1998**). (Sutcliffe *et al.*, **1998**) descrivono come creare e documentare scenari durante l'ingegneria dei requisiti. La riprogettazione dei processi aziendali è descritta in (Keen, **1991**) e (Tapscott e Caston, **1993**). Una struttura per BPR è data in (Davenport, **1993**).

La ricerca sull'elicitazione dei requisiti è finalizzata allo sviluppo di tecniche che superino i nostri limiti come esseri umani nel trasmettere informazioni. Una prima panoramica di questo tipo di problema è data in (Davis, **1982**). Un'indagine e una valutazione più recenti delle tecniche di elicitazione sono fornite in (Goguen e Linde, **1993**). Esempi di rapporti sull'esperienza sono forniti in (Sommerville *et al.*, **1994**); Coakes e Coakes (**2000**) adottano un approccio etnografico; e Beyer e Holtzblatt (**1995**) vedono l'analista come un apprendista dell'utente.

Lamsweerde (**2001**) offre un'ottima panoramica dell'ingegneria dei requisiti orientata agli obiettivi. (Mylopoulos *et al.*, **2001**) è un altro articolo di pionieri in questo settore. Darke e Shanks (**1996**) e Sommerville e Sawyer (**1997**) forniscono una buona panoramica dei punti di vista nel contesto dell'ingegneria dei requisiti.

(Leffingwell e Widrig, **2000**) è interamente dedicato alla gestione dei requisiti. Una delle prime discussioni sulla tracciabilità dei requisiti è (Gotel e Finkelstein, **1994**). L'analisi dello spazio progettuale è discussa in (Moran e Carroll, **1994**). L'approccio Domande, Opzioni,

Criteri (QOC) deriva da (MacLean *et al.*, **1991**). gIBIS, un sistema ipertestuale progettato per catturare le prime decisioni di progettazione, è descritto in (Conklin e Begeman, **1988**).

Il modello di Kano è discusso in (Kano, **1993**). La ricerca della creatività nell'ingegneria dei requisiti è ulteriormente sottolineata in (Robertson, **2002**), (Austin e Devin, **2003**) e (Maiden *et al.*, **2004**).

(Morisio *et al.*, **2002b**) forniscono una tassonomia dei prodotti COTS. Le procedure di selezione COTS sono discusse in (Maiden e Ncube, **1998**).

Esercizi

1.Quali sono i quattro principali tipi di attività nell'ingegneria dei requisiti?

2.Che cos'è l'elicitazione dei requisiti?

3.Qual è la differenza tra un modello concettuale implicito e uno esplicito?

4.In che senso la maggior parte delle tecniche di ingegneria dei requisiti sono di natura tayloriana?

5.Descrivere la tecnica di elicitazione dei requisiti chiamata "analisi del compito".

6.Descrivere la tecnica di elicitazione dei requisiti chiamata "analisi basata su scenari".

7.In quali circostanze l'etnografia è una tecnica praticabile di elicitazione dei requisiti?

8.Che cos'è l'ingegneria dei requisiti orientata agli obiettivi?

9.Come si possono rappresentare in punti di vista requisiti contrastanti?

10.Cosa significa MoSCoW?

11.Perché la distinzione tra le categorie di requisiti Attraente, Indispensabile e Unidimensionale nel modello di Kano è rilevante?

12.In che modo la presenza di componenti COTS influisce sull'ingegneria dei requisiti?

13.Perché la tracciabilità dei requisiti è importante?

14.Elencare e discutere i principali requisiti di qualità per un documento dei requisiti.

15.Elenca e discuti i principali svantaggi dell'uso del linguaggio naturale per specificare i requisiti.

16.♠Elabora una specifica dei requisiti per un sistema di cui sei stato coinvolto nello sviluppo, seguendo lo standard IEEE 830. Discuti le principali differenze tra la specifica originale e quella che hai scritto.

17.♡Quali sono le principali differenze nell'ambiente esterno di un sistema di automazione per ufficio e quello di un sistema embedded, come un sistema di controllo di ascensori. Quale impatto avranno queste differenze sulle tecniche di elicitazione dei requisiti da impiegare?

18.♡Per un sistema informativo d'ufficio, identificare diversi tipi di stakeholder. Riuscite a pensare ai modi in cui i requisiti di queste parti interessate potrebbero entrare in conflitto?

19.♡Affina il quadro nella **Figura 9.1 in** modo che rifletta la situazione in cui dobbiamo modellare esplicitamente sia la situazione lavorativa attuale che quella nuova.

20.♡Discutere i pro ei contro dei seguenti mezzi descrittivi per una specifica dei requisiti: linguaggio naturale completo, linguaggio naturale vincolato e un linguaggio pittorico (come UML).

21.♡Quale dei mezzi descrittivi menzionati nell'Esercizio 20 preferiresti per descrivere i requisiti di un sistema di automazione per ufficio? E quale per un sistema di controllo dell'ascensore?

22.♠Prendi il documento di specifica dei requisiti da un progetto in cui sei stato coinvolto e valutalo rispetto ai requisiti per tale documento come elencato nella **Sezione 9.2** (univocità, completezza, ecc.).

23.♡Come verificheresti i requisiti indicati nel documento dell'Esercizio 22? I requisiti sono verificabili per cominciare?

24.♠Come faresti per determinare i requisiti per un sistema di navigazione simile a un ipertesto per una biblioteca tecnica. Sia gli utenti che il personale della biblioteca hanno esperienza solo con i sistemi di recupero basati su parole chiave.

25.♡In qualità di analista coinvolto nello sviluppo di questo sistema di navigazione ipertestuale, discuti le possibili posizioni nelle dimensioni soggettivista-oggettivista e ordine-conflitto. Quali sono gli argomenti a favore e contro queste posizioni?

26.♠Scrivere una specifica dei requisiti per un sistema di navigazione ipertestuale.

27.♡Studiare la seguente specifica per un semplice formattatore di riga: L'input del programma è un flusso di caratteri la cui fine è segnalata da un carattere speciale di fine testo, ET. C'è esattamente un carattere ET in ogni flusso di input. I personaggi sono classificati come:

- caratteri di interruzione – BL (vuoto) e NL (nuova riga);
- l'indicatore di fine testo – ET;
- caratteri non divisibili – tutti gli altri.

Una *parola* è una sequenza non vuota di caratteri non di interruzione. Un'interruzione è una sequenza di uno o più caratteri di interruzione. Pertanto, l'input può essere visto come una sequenza di parole separate da interruzioni, con possibili interruzioni iniziali e finali, e terminanti con ET.

L'output del programma dovrebbe essere la stessa sequenza di parole dell'input, con l'eccezione che una parola sovradimensionata (cioè una parola contenente più di MAXPOS caratteri, dove MAXPOS è un numero intero positivo) dovrebbe causare un errore di uscita dal programma (cioè un variabile, Allarme, dovrebbe avere il valore TRUE). Fino al punto in cui si verifica un errore, l'output del programma dovrebbe avere le seguenti proprietà:

- Una nuova riga dovrebbe iniziare solo tra le parole e all'inizio del testo di output, se presente.
- Un'interruzione nell'input viene ridotta a un singolo carattere di interruzione nell'output.
- Il maggior numero possibile di parole dovrebbe essere posizionato su ogni riga (cioè tra caratteri NL successivi).
- Nessuna riga può contenere più di MAXPOS caratteri (parole e BL).

Identificare quanti più punti problematici è possibile in questa specifica. Confronta i tuoi risultati con quelli in (Meyer, **1985**).

28.♡Quali sono gli usi principali di una specifica dei requisiti. In che modo questi diversi usi influiscono sullo stile e sui contenuti di un documento dei requisiti?